# ozGUI / ozLib

A Python Toolkit for Solving the Ornstein–Zernike Equation

Theory, Formulas, and Usage Reference

August 25, 2026

**Abstract**

This document is the reference manual for `ozGUI` and `ozLib`, a Python re-implementation of SASfit's Ornstein–Zernike (OZ) equation solver (`src/sasfit_oz/` in the SASfit source tree). It documents, with exact mathematical formulas, every interaction potential, every closure relation, and every numerical solver algorithm currently implemented, together with the underlying Hankel-transform-based discretisation scheme and the two ways of using the toolkit: as an interactive Tkinter GUI (`oZgui.py`) and as a plain importable Python library (`ozLib.py`). Formulas throughout were ported and verified directly against SASfit's own C source (`src/sasfit_oz/sasfit_oz_solver.c`, `src/plugins/*`), and, where an independent literature source was available, cross-checked against it as well; deviations, known limitations, and one deliberately-excluded closure are documented explicitly rather than silently omitted. `oZgui.py` is specifically a from-scratch Tkinter port of SASfit's own built-in Tcl/Tk Ornstein–Zernike solver window; Section 8.1 cross- references the two GUIs' naming conventions directly against both source files, rather than assuming a 1:1 correspondence.

## Contents

**1 Overview** **2**

**2 The Ornstein–Zernike Equation and its Discretisation** **2**
2.1 The OZ equation . . . 2
2.2 Discrete grid: real space . . . 3
2.3 Discrete Hankel transform . . . 3
2.4 Grand-canonical thermodynamic quantities . . . 3

**3 Interaction Potentials** **3**
3.1 Hard Sphere . . . 4
3.2 Lennard-Jones . . . 4
3.3 Yukawa (screened Coulomb), with optional hard core . . . 4
3.4 Star polymer potential (Likos & Harreis) [20] . . . 4
3.5 Depletion potential (Oversteegen & Lekkerkerker) [21] . . . 5
3.6 DLVO (electrostatic double-layer + van der Waals) . . . 5
3.7 DLVO + hydration repulsion . . . 6
3.8 Fermi distribution model (Likos) . . . 6
3.9 Generalized Gaussian core model (GGCM-$n$) . . . 6
3.10 Hard sphere + 3 additive Yukawa tails . . . 6
3.11 Ionic microgel (Likos et al.) . . . 6
3.12 Piecewise-constant hard-sphere shoulder/well (3 segments) . . . 6
3.13 Penetrable sphere model (Likos, Watzlawek, Löwen) [22] . . . 7

3.14 Soft sphere (inverse power law) . . . 7
3.15 Parabolic (Hertzian) sphere . . . 7
3.16 Square well . . . 7
3.17 Sticky hard sphere (Baxter's adhesive limit) . . . 7

**4 Closure Relations** **8**
4.1 Percus–Yevick (PY) . . . 8
4.2 Hypernetted Chain (HNC) . . . 8
4.3 Reference HNC (RHNC) . . . 8
4.4 Modified HNC (MHNC) . . . 8
4.5 Mean Spherical Approximation (MSA) . . . 8
4.6 Rescaled MSA (RMSA) . . . 9
4.7 Modified MSA (mMSA) . . . 9
4.8 "Symmetric" MSA (SMSA) . . . 9
4.9 Rogers–Young (RY) [8] . . . 9
4.10 HMSA (Zerah–Hansen) [9] . . . 9
4.11 Verlet [10, 11] . . . 9
4.12 Martynov–Sarkisov (MS) [12] . . . 10
4.13 Vompe–Martynov (VM) [13] . . . 10
4.14 BPGG (Ballone–Pastore–Galli–Gazzillo) [14] . . . 10
4.15 CJVM (Charpentier–Jakse extension of VM) [15, 13] . . . 10
4.16 BB (Bomont–Bretonnet) [16] . . . 10
4.17 Kovalenko–Hirata (KH) [17] . . . 10
4.18 Duh–Haymet (DH) [18] . . . 11
4.19 Choudhury–Ghosh (CG) [19] . . . 11
4.20 ZSEP (Lee) [24, 25] — hard spheres only . . . 11
4.21 Euler–Rahman (EuRah) — not offered . . . 12
4.22 Practical guidance: which closure for which potential . . . 13
4.23 Literature validation against Monte Carlo: known accurate and inaccurate combinations . . . 14

**5 Analytical and Semi-Analytical Cross-Validation Tools** **17**
5.1 Percus–Yevick hard sphere (Wertheim–Thiele) . . . 17
5.2 Baxter sticky hard sphere (idealised and finite-well) . . . 17
5.3 Sharma–Sharma square well . . . 17
5.4 Rescaled MSA (Hayter–Penfold) . . . 17
5.5 One-Yukawa MSA (Blum–Høye) . . . 18
5.6 ORPA (Pini–Parola–Reatto) for square wells . . . 18
5.7 RFA (Rational Function Approximation) . . . 18

**6 Numerical Solver Algorithms** **19**
6.1 Picard iteration . . . 19
6.2 Anderson acceleration (hand-written) . . . 19
6.3 scipy Anderson / scipy Newton–Krylov . . . 19
6.4 Biggs–Andrews . . . 19
6.5 SUNDIALS KINSOL: Newton–Krylov (GMRES/FGMRES/TFQMR) . . . 19
6.6 SUNDIALS KINSOL: Fixed-Point (Anderson), `KIN_FP` . . . 20
6.7 Empirical solver comparison . . . 20

**7 The `ozLib` Python Library** **21**
7.1 Basic usage . . . 21
7.2 Function signature . . . 21

7.3 Example: potential with arguments and a parametrised closure . . . . . 22
7.4 Example: long-range potential, extended grid . . . . . 22

**8 The `ozGUI` Application** **22**
8.1 Relationship to SASfit's own Tcl GUI, and naming-convention differences . . . . 22

**9 Worked Examples: Numerical vs. Analytical Validation** **23**
9.1 Hard-sphere Percus–Yevick: full validation across $\varphi$ . . . . . 24
9.2 Closure comparison at high density . . . . . 24
9.3 Charged systems: RMSA and the MSA contact-value problem . . . . . 25
9.4 Behaviour near difficult/critical parameter regimes . . . . . 27
9.5 Speed . . . . . 28

# 1 Overview

`ozGUI/ozLib` solves the Ornstein–Zernike (OZ) integral equation for a single-component, isotropic fluid of spherically symmetric particles, given

- a pair interaction potential $U(r)$ (Section 3),
- a closure relation connecting the direct correlation function $c(r)$ to the total correlation function $h(r) = g(r) - 1$ (Section 4),
- a volume fraction $\varphi$, and
- a numerical solver algorithm that finds the self-consistent fixed point of the resulting nonlinear system (Section 6).

The output is the pair correlation function $g(r)$, the direct correlation function $c(r)$, the static structure factor $S(q)$, and several auxiliary quantities (the cavity function $y(r)$, the bridge function $B(r)$, the indirect correlation function $\Gamma(r) = h(r) - c(r)$, and the reduced potential $U(r)/k_BT$).

Two front ends share the exact same computational core (`ozLib.py`), so results obtained through either are identical by construction:

- `oZgui.py` — an interactive Tkinter application with potential/closure/solver dropdowns, a 9-tab plot panel, run history, and Save/Load/ASCII/CSV/Excel/PNG export (Section 8).
- `ozLib.py` — a plain Python function, `ozLib.solve(...)`, usable directly in scripts without any GUI dependency (Section 7).

# 2 The Ornstein–Zernike Equation and its Discretisation

## 2.1 The OZ equation

For an isotropic, single-component fluid the Ornstein–Zernike equation reads, in real space,

$$h(r) = c(r) + \rho \int c(|\mathbf{r} - \mathbf{r}'|)\, h(r')\, d\mathbf{r}', \tag{1}$$

where $\rho$ is the number density and the convolution is over all space. Introducing the indirect correlation function $\Gamma(r) \equiv h(r) - c(r)$, and Fourier–Bessel (Hankel) transforming (exploiting isotropy to reduce the 3D Fourier transform to a 1D sine transform), the OZ equation becomes purely algebraic in reciprocal space,

$$\hat{\Gamma}(q) = \frac{\hat{c}(q)}{1 - \rho\,\hat{c}(q)} - \hat{c}(q), \tag{2}$$

where $\hat{f}(q)$ denotes the (3D, spherically-symmetric) Fourier transform of $f(r)$. The OZ equation alone relates two unknowns, $c(r)$ and $h(r)$ (or equivalently $\Gamma(r)$); a second, independent relation — the *closure* — is required to close the system. Every closure implemented here has the general form

$$c(r) = f\big(\Gamma(r), U(r)\big) \tag{3}$$

for some function $f$ specific to the closure (Section 4). The full problem is therefore a fixed-point problem in $\Gamma(r)$ (or, in an equivalent formulation used by some solvers here, in $c(r)$ and $h(r)$ jointly): starting from a trial $\Gamma(r)$, the closure gives $c(r)$, the OZ equation (Eq. 2) gives a new $\Gamma(r)$, and a solver algorithm (Section 6) drives this map to self-consistency.

### 2.2 Discrete grid: real space

The real-space grid is a plain equal-spaced grid of $N =$ `numberOfRadialSamplingPoints` points,

$$r_i = i\,\Delta r, \qquad i = 1, 2, \ldots, N, \tag{4}$$

starting at $\Delta r$ rather than 0 (division by $r$ appears in the Hankel transform below, so $r = 0$ is avoided by construction rather than special-cased). The hard-core diameter $\sigma$ is placed at grid index `hardSphereDiameterInPoints` (called $n_\sigma$ below), i.e.

$$\sigma = n_\sigma\,\Delta r, \qquad \text{total range } r_{\max} = (N-1)\Delta r = \frac{N-1}{n_\sigma}\,\sigma. \tag{5}$$

Both $N$ and $n_\sigma$ are configurable (constructor keyword arguments `numberOfRadialSamplingPoints`, `hardSphereDiameterInPoints` on every solver class, and identically-named arguments on `ozLib.solve()`); the historical defaults are $N = 4096$, $n_\sigma = 100$, giving $r_{\max} \approx 41\,\sigma$ and $\Delta r = 0.01\,\sigma$. Increasing $N$ at fixed $n_\sigma$ extends the range without losing near-contact resolution — important for slowly-decaying, long-range potentials (e.g. small-$\kappa$ Yukawa/DLVO tails), where a too-short grid truncates the potential before it has actually decayed to zero, distorting $S(q)$ at low $q$ (this was verified directly: a $4\times$ extension of the default grid reduced a Yukawa potential's residual value at the grid edge from $\mathcal{O}(10^{-4})\,k_BT$ to $\mathcal{O}(10^{-10})\,k_BT$, and changed $S(q \to 0)$ by about 33%).

### 2.3 Discrete Hankel transform

For an isotropic function $f(r)$ the 3D Fourier transform reduces to

$$\hat{f}(q) = \frac{4\pi}{q}\int_0^\infty r\,f(r)\,\sin(qr)\,dr, \tag{6}$$

which is implemented here as a type-I discrete sine transform (DST) of $r_i f(r_i)$, following the same construction documented in SASfit's own theory notes (equations 5.18/5.19 there):

$$\hat{f}(q_k) = \frac{2\pi\Delta r^2}{\Delta q}\cdot\frac{\text{DST-I}\big[r_i f(r_i)\big]_k}{k}, \qquad \Delta q = \frac{\pi}{(N+1)\Delta r}, \qquad q_k = k\,\Delta q. \tag{7}$$

The inverse transform is the same operation up to a normalisation factor ($\Delta q^3/[(2\pi)^3\Delta r^3]$), since the Hankel transform pair is (up to constants) self-inverse. Both directions are implemented once, in `OZfixpointOperator.hankelTransform()`/`inverseHankelTransform()`, and shared by every closure and every solver.

### 2.4 Grand-canonical thermodynamic quantities

For the hard-sphere/Percus–Yevick case, the exact grand-canonical compressibility-route structure factor at $q = 0$ is available in closed form (used as an internal consistency check, e.g. in RMSA's own bisection, see Section 4.6):

$$S(q{=}0) = \frac{(1-\varphi)^4}{(1+2\varphi)^2}. \tag{8}$$

## 3 Interaction Potentials

Every potential is specified through its Boltzmann factor $\exp[-U(r)/k_BT]$ (stored as `boltzmannOfP2Ppotential`) evaluated on the grid described above; all energies are in $k_BT$ units and all lengths in units where the hard-core diameter is $\sigma = n_\sigma\Delta r$. Potentials that support the repulsive/attractive potential

split needed by switching-function closures (Rogers–Young, HMSA, and the Gstar-based bridge closures of Section 4) additionally populate the arrays `repulsivePartOfP2Ppotential` and `attractivePartOfP2Ppotential`. All 18 potentials below are available via `setPotentialByName(name, *args)` or directly as `set<Name>Potential(...)` – in 17 subsections, not 18, since Section 3.4 covers both of the star polymer potential's two distinct setters (`setStarPolymerHighFPotential()`, `setStarPolymerLowFPotential()`) together, one formula each.

*Note on counting, and on a genuine fix this made:* the GUI's own potential dropdown (Table 2, Section 8.1) lists 18 named entries. An earlier version of this port had only 17 setters – `setStarPotential()` implemented only the star polymer potential's $f \geq 10$ formula, silently applied regardless of functionality, with no $f \leq 10$ counterpart at all, unlike the original Tcl GUI's two separate `StarPolymer (f>10)`/ `StarPolymer (f<10)` dropdown entries. Checking `sasfit_oz_potential_star_Likos.c` directly (rather than assuming the single existing formula covered every $f$) found this is a genuinely different second formula (`U_Star2`, Section 3.4), not just the same one evaluated at a smaller argument – now added as `setStarPolymerLowFPotential()`, matching the Tcl GUI's count exactly. `Depl-Sph-Sph`/`Depl-Sph-Discs`/`Depl-Sph-Rods` similarly appear as three separate GUI choices against one Depletion subsection here – checked the same way the star polymer case was: confirmed to be a similarly-missing split (see Section 3.5's own **Known gap** note), not a single shared formula with different constant substitutions, though unlike star polymer this one was found but not fixed in this pass.

### 3.1 Hard Sphere

$$\exp[-U(r)/k_BT] = \begin{cases} 0 & r < \sigma \\ 1 & r \geq \sigma. \end{cases} \tag{9}$$

### 3.2 Lennard-Jones

$$U(r)/k_BT = 4\varepsilon \left[ \left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6} \right], \tag{10}$$

with $\varepsilon$ in $k_BT$ units. The potential is split at its minimum ($r_{\min} = 2^{1/6}\sigma$) into a repulsive part ($r < r_{\min}$) and an attractive part ($r \geq r_{\min}$), for use by switching-function closures.

### 3.3 Yukawa (screened Coulomb), with optional hard core

$$U(r)/k_BT = K \frac{\sigma}{r} e^{-r/\lambda}, \tag{11}$$

where $\lambda$ is the screening length (in units of $\sigma$) and $K$ the contact interaction strength; a hard core can optionally be added (`doAddHardSphere`), in which case the exponential and $1/r$ factors are shifted independently as in SASfit's own construction, and the potential is set to $\infty$ (Boltzmann factor 0) for $r < \sigma$.

### 3.4 Star polymer potential (Likos & Harreis) [20]

Two genuinely different formulas, one for each of two functionality regimes, matching `U_Star1` and `U_Star2` in `sasfit_oz_potential_star_Likos.c` respectively. This port calls them `setStarPolymerHighFPotential()` and `setStarPolymerLowFPotential()`; an earlier version only had the first of these, silently applying it regardless of $f$.

**High functionality,** $f \geq 10$ (exponential decay outside the core):

$$U(r)/k_BT = \begin{cases} \dfrac{5}{18} f^{3/2} \left[ -\ln\dfrac{r}{\sigma} + \dfrac{1}{1+\sqrt{f}/2} \right] & r \leq \sigma \\ \dfrac{5}{18} f^{3/2} \dfrac{1}{1+\sqrt{f}/2} \dfrac{\sigma}{r} \exp\left[ -\dfrac{\sqrt{f}(r-\sigma)}{2\sigma} \right] & r > \sigma. \end{cases} \tag{12}$$

**Low functionality,** $f \leq 10$ (Gaussian decay outside the core, with its own $\tau(f)$ interpolation):

$$\tau(f) = \tau_2 + \frac{\tau_5 - \tau_2}{3} f, \qquad \tau_2 = \frac{1.03}{\sigma}, \quad \tau_5 = \frac{1.12}{\sigma}, \tag{13}$$

$$U(r)/k_BT = \begin{cases} \dfrac{5}{18} f^{3/2} \left[ -\ln\dfrac{r}{\sigma} + \dfrac{1}{2(\tau\sigma)^2} \right] & r \leq \sigma \\ \dfrac{5}{18} f^{3/2} \dfrac{1}{2(\tau\sigma)^2} \exp\left[ -\tau^2(r^2 - \sigma^2) \right] & r > \sigma. \end{cases} \tag{14}$$

Here $f =$ `NumberOfArms` is the number of arms of the star polymer (the same argument name on both setters). The two branches do *not* meet smoothly at $f = 10$ – checked directly: ∼28% relative difference in $U(r = \sigma^+)$, about 30× apart by $r = 2\sigma$. This is a genuine property of the original paper's own two-regime fit (each branch is only claimed accurate within its own stated range), not a bug in either formula or this port; $f = 10$ is where Likos & Harreis recommend switching approximations, not a point the two forms were designed to agree at exactly. A caller choosing between the two should therefore pick whichever branch matches their actual $f$, not assume either extrapolates sensibly across the boundary.

### 3.5 Depletion potential (Oversteegen & Lekkerkerker) [21]

Two-sphere overlap-volume depletion potential between big spheres (diameter $\sigma_1 = \sigma$, the species this OZ solve is for) induced by small depletant spheres (diameter $\sigma_2 =$ `sigmaRatio`$\cdot\sigma_1$, number density $\rho_2$ set via the depletant volume fraction `phi2`):

$$U(r)/k_BT = -\rho_2 \frac{\pi\sigma_1^3(1+r_t)^3}{6} \left[ 1 - \frac{3r}{2(1+r_t)\sigma_1} + \frac{r^3}{2(1+r_t)^3\sigma_1^3} \right], \qquad \sigma_1 \leq r < \sigma_1 + \sigma_2, \tag{15}$$

with $r_t = \sigma_2/\sigma_1$, and $U = 0$ for $r \geq \sigma_1 + \sigma_2$; hard core for $r < \sigma_1$.

**Known gap, found while cross-checking the star polymer split of Section 3.4 prompted a closer look at this file too:** `sasfit_oz_potential_depletion.c` actually contains *four* distinct formulas, not one – the generic `U_Depletion` above, plus `U_DepletionOfSpheresBySpheres`, `U_DepletionOfSpheresByDiscs`, and `U_DepletionOfSpheresByRods`, matching the Tcl GUI's own three named dropdown entries (`Depl-Sph-Sph`/`Depl-Sph-Discs`/ `Depl-Sph-Rods`, Table 2, Section 8.1) – each geometrically distinct (sphere depleted by spheres/discs/rods respectively, the disc and rod cases involving genuinely different overlap-volume geometry, e.g. an arcsin term for discs) and parametrised differently from `U_Depletion` (`sigma_large`, `sigma_small`/`D`/`L`, and a number density `nb` directly, rather than `sigma1`/`sigma2`/a volume fraction `PHI2`). This Python port currently only has the generic `U_Depletion` form – unlike the star polymer fix of Section 3.4, this was found but deliberately *not* fixed in this pass (out of the scope of what was asked, and a larger, riskier addition: three new geometries to verify, not one), and is recorded here so it isn't silently lost.

### 3.6 DLVO (electrostatic double-layer + van der Waals)

$$U(r)/k_BT = U_{\mathrm{el}}(r) + U_{\mathrm{vdW}}(r), \qquad r > \sigma, \tag{16}$$

$$U_{\mathrm{el}}(r) = \frac{L_B Z^2}{(1+\kappa a)^2}\,\frac{e^{-\kappa(r-\sigma)}}{r}, \qquad U_{\mathrm{vdW}}(r) = -\frac{A}{6}\left[\frac{2a^2}{r^2-4a^2} + 2\left(\frac{a}{r}\right)^2 + \ln\!\left(1-4\left(\frac{a}{r}\right)^2\right)\right], \tag{17}$$

where $a = \sigma/2$, $\kappa$ the inverse Debye screening length, $Z$ the macro-ion charge, $L_B$ the Bjerrum length, and $A$ the effective Hamaker constant. $U_{\mathrm{vdW}}$'s genuine divergence at $r = \sigma$ is excluded from the computation itself (not merely masked afterward), since the grid places a real point at exactly $r = \sigma$.

### 3.7 DLVO + hydration repulsion

As Section 3.6, plus a short-range exponential hydration term,

$$U(r)/k_BT = U_{\mathrm{DLVO}}(r) - 2\,G_{HY}\,e^{-(r-\sigma)/D_H}, \qquad r > \sigma, \tag{18}$$

with hydration strength $G_{HY}$ and decay length $D_H$.

### 3.8 Fermi distribution model (Likos)

$$U(r)/k_BT = \varepsilon\,\frac{1+e^{-\sigma/\xi}}{1+e^{(r-\sigma)/\xi}}. \tag{19}$$

The limit $\xi \to 0$ reduces to the penetrable sphere model (below), and is handled as a special case when $\xi = 0$ is passed exactly.

### 3.9 Generalized Gaussian core model (GGCM-$n$)

$$U(r)/k_BT = \varepsilon\,\exp\!\left[-\alpha\left(\frac{r}{\sigma}\right)^n\right]. \tag{20}$$

### 3.10 Hard sphere + 3 additive Yukawa tails

$$U(r)/k_BT = -\frac{\sigma}{r}\sum_{i=1}^{3} K_i\,\exp\!\left[-\frac{r-\sigma}{\sigma\,\lambda_i}\right], \qquad r \geq \sigma, \tag{21}$$

with hard core for $r < \sigma$; each $(K_i, \lambda_i)$ pair may be set to zero to recover fewer effective Yukawa terms.

### 3.11 Ionic microgel (Likos et al.)

A multi-term analytical potential for a charged microgel sphere of charge $Z$, dielectric constant $ED$, and inverse screening length $\kappa_\Pi/\sigma$ inside the gel, following Likos et al. (2011); the full piecewise expression (different polynomial/hyperbolic forms inside vs. outside the gel radius) is implemented exactly as in `sasfit_oz_potential_ionic_microgel.c` and is reproduced verbatim in the source rather than restated here, since it spans several auxiliary terms without a single compact closed form.

### 3.12 Piecewise-constant hard-sphere shoulder/well (3 segments)

$$U(r)/k_BT = \begin{cases} \varepsilon_1 & \sigma \leq r < \sigma+\delta_1 \\ \varepsilon_2 & \sigma+\delta_1 \leq r < \sigma+\delta_1+\delta_2 \\ \varepsilon_3 & \sigma+\delta_1+\delta_2 \leq r < \sigma+\delta_1+\delta_2+\delta_3 \\ 0 & \text{otherwise (outside all three steps)}, \end{cases} \tag{22}$$

hard core for $r < \sigma$; each $\varepsilon_j$ may be positive (shoulder) or negative (well). This is the general $n = 3$ piecewise-constant family used as an independent cross-check for the RFA analytical solver (Section 5.7).

### 3.13 Penetrable sphere model (Likos, Watzlawek, Löwen) [22]

$$U(r)/k_BT = \begin{cases} \varepsilon & r \le \sigma \\ 0 & r > \sigma. \end{cases} \tag{23}$$

### 3.14 Soft sphere (inverse power law)

$$U(r)/k_BT = \varepsilon \left(\frac{\sigma}{r}\right)^n. \tag{24}$$

### 3.15 Parabolic (Hertzian) sphere

$$U(r)/k_BT = \begin{cases} \varepsilon \left(1 - \dfrac{r}{\sigma}\right)^{5/2} & r \le \sigma \\ 0 & r > \sigma. \end{cases} \tag{25}$$

### 3.16 Square well

$$U(r)/k_BT = \begin{cases} \infty & r < \sigma \\ \varepsilon & \sigma \le r \le \sigma + \delta \\ 0 & r > \sigma + \delta, \end{cases} \tag{26}$$

with well width $\delta$ (well outer range $\lambda = 1 + \delta/\sigma$ in the notation of Section 5.3); $\varepsilon < 0$ for an attractive well, $\varepsilon > 0$ for a repulsive shoulder.

### 3.17 Sticky hard sphere (Baxter's adhesive limit)

$$U(r)/k_BT = \begin{cases} \infty & r < \sigma \\ \ln\left(\dfrac{12\,\tau\,\delta}{\sigma + \delta}\right) & \sigma \le r \le \sigma + \delta \\ 0 & r > \sigma + \delta, \end{cases} \tag{27}$$

using Baxter's stickiness parameter $\tau$ (small $\tau$ = sticky, large $\tau$ = hard-sphere limit) and a finite well width $\delta$ — the same $\tau$ convention as SASfit's own `robertus_shs` plugin, giving a natural point of comparison between this (monodisperse, OZ-based) solve and that (polydisperse, PY-only, multicomponent) one.

# 4 Closure Relations

Every closure below expresses $c(r)$ as a function of $\Gamma(r)$ (and, for the switching-function and Gstar-based closures, the potential's own repulsive/attractive split) and is dispatched through `OZfixpointOperator.update_c()`. Formulas were cross-checked directly against SASfit's own `sasfit_oz_solver.c case` statements (not re-derived from a textbook), using that file's own notation: $\mathrm{EN}(r) = \exp[-U(r)/k_BT]$, $G(r) = \Gamma(r)$. Where a closure needs an extra scalar parameter (shared storage slot `self.alpha`, matching the C code's own `sBPGG/aCJVM/fBB #define`s onto the same underlying variable), this is noted explicitly. **20 closures are offered** – 19 general-purpose ones plus ZSEP (Section 4.20, hard spheres only, self-fitting its own three free parameters rather than taking any of them from the caller); a 21st (Euler–Rahman) exists in the source but is deliberately excluded from both the GUI and `ozLib.solve()`'s validated closure list – see Section 4.21 for why.

## 4.1 Percus–Yevick (PY)

$$c(r) = \mathrm{EN}(r)\big[1 + \Gamma(r)\big] - \Gamma(r) - 1. \tag{28}$$

Equivalent to the standard PY form $c(r) = \big(e^{-U/k_BT} - 1\big)y(r)$ with cavity function $y(r) = g(r)e^{U/k_BT}$.

## 4.2 Hypernetted Chain (HNC)

$$c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)} - \Gamma(r) - 1. \tag{29}$$

## 4.3 Reference HNC (RHNC)

Needs a reference-system sub-solve: a bare hard sphere of the same $\sigma$ is first solved with the Martynov–Sarkisov closure (Section 4.12), giving reference cavity/indirect-correlation functions $g_0(r)$, $\Gamma_0(r)$. Writing the actual potential's perturbation as $\Delta U(r)/k_BT = -\ln \mathrm{EN}(r)$ (zero inside the reference hard core), the closure is

$$c(r) = g_0(r)\, \exp\big[\big(\Gamma(r) - \Gamma_0(r)\big) - \Delta U(r)/k_BT\big] - \Gamma(r) - 1. \tag{30}$$

This reduces to the same hard-core exclusion every other hard-sphere closure uses automatically, since $g_0(r) \equiv 0$ inside the reference hard core. Orchestrated by `OZsolver.solveRHNC()`.

## 4.4 Modified HNC (MHNC)

HNC plus a fixed, purely analytical Percus–Yevick hard-sphere bridge function $B_{\mathrm{PY}}(r;\eta)$ (no reference-system sub-solve needed, unlike RHNC, since $B_{\mathrm{PY}}$ has a closed form):

$$c(r) = \mathrm{EN}(r)\, \exp\big[\Gamma(r) + B_{\mathrm{PY}}(r;\eta)\big] - \Gamma(r) - 1, \tag{31}$$

with the packing-fraction-like parameter $\eta$ supplied by the user (`doMHNCclosure(eta)`).

## 4.5 Mean Spherical Approximation (MSA)

$$c(r) = \begin{cases} -\big[\Gamma(r) + 1\big] & r < \sigma \quad \text{(forces } g(r)\text{=0)} \\ -U(r)/k_BT = \ln \mathrm{EN}(r) & r \geq \sigma. \end{cases} \tag{32}$$

### 4.6 Rescaled MSA (RMSA)

Identical formula to MSA, but the inside/outside switch is driven by an *apparent* hard-core gate $g_{4g}(r) \in \{0, 1\}$ rather than the bare potential's own hard core:

$$c(r) = \begin{cases} -\big[\Gamma(r) + 1\big] & g_{4g}(r) = 0 \\ \ln \mathrm{EN}(r) & g_{4g}(r) = 1. \end{cases} \tag{33}$$

`OZsolver.solveRMSA()` first solves plain MSA; if the resulting $g(\sigma^+) < 0$ (an unphysical result plain MSA can give for strongly-coupled systems), it re-solves with the apparent hard core bisected outward, grid point by grid point, until $g$ at the new apparent contact point is non-negative. This is the same Hayter–Penfold-style rescaling idea underlying Section 5.4's semi-analytical RMSA solver, applied here inside the general numerical OZ solve instead of via the closed-form Hayter–Penfold machinery.

### 4.7 Modified MSA (mMSA)

As MSA, but using the Mayer function outside the core instead of the bare potential:

$$c(r) = \begin{cases} -\big[\Gamma(r) + 1\big] & r < \sigma \\ \mathrm{EN}(r) - 1 & r \geq \sigma. \end{cases} \tag{34}$$

### 4.8 "Symmetric" MSA (SMSA)

Using $\Gamma^*(r) = \Gamma(r) - U_{\mathrm{attr}}(r)/k_BT$ (the attractive-part-subtracted indirect correlation function, shared with several closures below):

$$c(r) = \exp\big[U_{\mathrm{attr}}(r)/k_BT\big]\big[\Gamma^*(r) + 1\big] - \Gamma(r) - 1. \tag{35}$$

### 4.9 Rogers–Young (RY) [8]

Interpolates between PY ($\alpha \to \infty$) and HNC ($\alpha \to 0$) via a switching function $f(r) = 1 - e^{-\alpha r}$:

$$c(r) = \mathrm{EN}(r)\left[1 + \frac{e^{f(r)\Gamma(r)} - 1}{f(r)}\right] - \Gamma(r) - 1, \tag{36}$$

with $\alpha$ supplied by the user (`doRYclosure(alpha)`).

### 4.10 HMSA (Zerah–Hansen) [9]

Same switching-function structure as RY, but using $\Gamma^*(r)$ in the switching term and the full Boltzmann factor split into repulsive/attractive parts:

$$c(r) = \exp\big[-U_{\mathrm{rep}}(r)/k_BT\big]\left[1 + \frac{e^{f(r)\Gamma^*(r)} - 1}{f(r)}\right] - \Gamma(r) - 1, \tag{37}$$

with $\alpha$ (in $f(r) = 1 - e^{-\alpha r}$) supplied by the user (`doHMSAclosure(alpha)`).

### 4.11 Verlet [10, 11]

Empirical bridge function, no reference system or extra parameter:

$$B(r) = \frac{-\Gamma(r)^2}{2\big[1 + 0.8\,\Gamma(r)\big]}, \qquad c(r) = \mathrm{EN}(r)\, e^{\Gamma(r) + B(r)} - \Gamma(r) - 1. \tag{38}$$

### 4.12 Martynov–Sarkisov (MS) [12]

$$B(r) = \mathrm{sgn}(1{+}2\Gamma)\sqrt{|1+2\Gamma(r)|} - \Gamma(r) - 1, \qquad c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{39}$$

Used standalone, and as the reference-system closure for RHNC (Section 4.3).

### 4.13 Vompe–Martynov (VM) [13]

As MS, but using $\Gamma^*(r)$ in place of $\Gamma(r)$ inside the square root:

$$B(r) = \mathrm{sgn}(1{+}2\Gamma^*)\sqrt{|1+2\Gamma^*(r)|} - \Gamma^*(r) - 1, \qquad c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{40}$$

### 4.14 BPGG (Ballone–Pastore–Galli–Gazzillo) [14]

Power-law bridge function with user parameter $s$ (`doBPGGclosure(s)`, $s \neq 0$; SASfit's own `sBPGG`):

$$B(r) = \mathrm{sgn}(1{+}s\Gamma)\,|1 + s\,\Gamma(r)|^{1/s} - \Gamma(r) - 1, \qquad c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{41}$$

### 4.15 CJVM (Charpentier–Jakse extension of VM) [15, 13]

User parameter $a$ (`doCJVMclosure(a)`, $a \neq 0$; SASfit's own `aCJVM`):

$$B(r) = \frac{1}{2a}\Big[\mathrm{sgn}(1{+}4a\Gamma^*)\sqrt{|1+4a\,\Gamma^*(r)|} - 1 - 2a\,\Gamma^*(r)\Big] - \Gamma^*(r) - 1, \tag{42}$$

$$c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{43}$$

### 4.16 BB (Bomont–Bretonnet) [16]

User parameter $f$ (`doBBclosure(f)`; SASfit's own `fBB`):

$$B(r) = \mathrm{sgn}\big(1{+}2\Gamma^*{+}f(\Gamma^*)^2\big)\sqrt{\big|1+2\Gamma^*(r)+f\,\Gamma^*(r)^2\big|} - \Gamma^*(r) - 1, \tag{44}$$

$$c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{45}$$

### 4.17 Kovalenko–Hirata (KH) [17]

$$d(r) = \Gamma(r) - U(r)/k_BT = \Gamma(r) + \ln \mathrm{EN}(r), \qquad g(r) = \begin{cases} \mathrm{EN}(r)\, e^{\Gamma(r)} & d(r) \le 0 \\ 1 + d(r) & d(r) > 0, \end{cases} \tag{46}$$

$$c(r) = g(r) - \Gamma(r) - 1. \tag{47}$$

This is the real literature formula (equivalently, $b(r) = [\ln(1 + d(r)) - d(r)]\,\theta(d(r))$, matching Kovalenko & Hirata's own closure [17] as reproduced independently in Pihlajamaa & Janssen's (2024) closure-comparison table [2]) – identical to HNC wherever $d(r) \le 0$, and linearised (bounded, non-exponential in $\Gamma(r)$) wherever $d(r) > 0$. This is the actual mechanism behind KH's well-documented good numerical convergence [28]: HNC's own $c(r)$ grows without bound as $\Gamma(r)$ grows, whereas KH's linear branch, $c(r) = \ln \mathrm{EN}(r) = -U(r)/k_BT$, does not depend on $\Gamma(r)$ at all, and so cannot diverge that way.

**Remark 1.** *An earlier version of this port matched SASfit's own* `case KH:` *block in* `sasfit_oz_solver.c` *exactly, rather than this formula: that C code computes the piecewise-linearised bridge function above too, but only as a separate diagnostic* `BRIDGE[i]` *output that is never fed back into $c(r)$ in that same closure branch – so that earlier port was, in effect, plain HNC under a different name (confirmed then: identical $g(r)$ to machine precision for every case tested). This has now*

*been corrected to the formula above, chosen and verified directly against the literature rather than against that specific quirk of SASfit's own C source.*

*Verified three ways.* Continuity*: the two branches give identical $c(r)$ and $g(r)$ at the switch boundary $d(r) = 0$ (checked to $10^{-10}$), so the correction introduces no discontinuity.* Hard-core exclusion*: where* $\mathrm{EN}(r) = 0$, $\ln \mathrm{EN}(r) \to -\infty$ *forces* $d(r) \le 0$, *selecting the HNC branch, which reduces to the same* $c(r) = -(\Gamma(r) + 1)$ *hard-core exclusion PY/HNC/MSA all use – confirmed numerically for* `HardSphere`*, not just algebraically.* Full self-consistent solve*: for hard spheres at* $\varphi = 0.3$ *(Picard iteration, this project's own solver), the corrected KH gives* $g(\sigma^+) = 2.360$*, close to the exact PY value (*2.356*) and clearly different from plain HNC's own* 2.845 *at the same state point – expected, since a bare hard-sphere potential has* $\mathrm{EN}(r) = 1$ *exactly for* $r > \sigma$*, so KH's linear branch there gives* $c(r) = \ln(1) = 0$*, the same value PY's own formula happens to give outside contact for a hard sphere. At* $\varphi = 0.45$*, neither the corrected KH, HNC, PY, nor the previous (HNC-identical) KH implementation converges within 2000 plain-Picard iterations – the same already-documented Picard limitation at this density (Section 9.4), not a KH-specific issue: all four closures hit the iteration cap with comparably-sized residuals, so this is not evidence either for or against KH's own convergence properties specifically; a fair test of that specific literature claim would need an accelerated solver (Section 6), which was not attempted here.*

### 4.18 Duh–Haymet (DH) [18]

A rational-function bridge approximation:

$$B(r) = \frac{-\Gamma^*(r)^2}{2\left[1 + \dfrac{5\,\Gamma^*(r) + 11}{7\,\Gamma^*(r) + 9}\,\Gamma^*(r)\right]}, \qquad c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{48}$$

Independently verified against Pihlajamaa & Janssen (2024) [2], Table 1: $b(r) = -\frac{1}{2}\gamma^2(r)\big[1 + \gamma(r)(5\gamma(r) + 11)/(7\gamma(r) + 9)\big]^{-1}$, an exact match. *Known gap:* SASfit's own `case DH:` additionally special-cases the $\Gamma^*$ formula specifically when the active potential is Lennard-Jones (a narrower numerical-stability tweak for that potential's own $r^{-12}$ term, not a different closure formula); this special case is not replicated here.

### 4.19 Choudhury–Ghosh (CG) [19]

A piecewise bridge function with a density-dependent coefficient:

$$B(r) = \begin{cases} \dfrac{-\frac{1}{2}\,\Gamma^*(r)^2}{1 + \zeta(\varphi)\,\Gamma^*(r)} & \Gamma^*(r) > 0 \\ -\frac{1}{2}\,\Gamma^*(r)^2 & \Gamma^*(r) \le 0, \end{cases} \qquad \zeta(\varphi) = 1.0175 - 0.275\,\frac{6\varphi}{\pi}, \tag{49}$$

$$c(r) = \mathrm{EN}(r)\, e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \tag{50}$$

Independently verified against Pihlajamaa & Janssen (2024) [2], Table 1: $\zeta(\rho) = 1.0175 - 0.275\,\rho\sigma^3$, and since $\rho\sigma^3 = 6\varphi/\pi$ for spheres, this is an exact match (including the same $\Gamma^* \le 0$ piecewise branch).

### 4.20 ZSEP (Lee) [24, 25] — hard spheres only

Unlike every closure above, ZSEP is not a fixed formula the caller parametrises: it self-fits its own three free parameters ($\zeta, \varphi_Z, \alpha$ – `zsep_zeta`/`zsep_phi`/ `zsep_alpha`, distinct from the system's own volume fraction $\varphi$ despite the similar name) against three *exact* conditions derived from

the Carnahan–Starling hard-sphere equation of state, following Lee (1995) [24], Eqs. (2.6)–(2.10) and (3.7)–(3.9). Using $\Gamma^*(r) = \Gamma(r) + \frac{\rho}{2}\mathrm{MAYER}(r)$, $\mathrm{MAYER}(r) = \mathrm{EN}(r) - 1$:

$$B(\Gamma^*) = -\tfrac{1}{2}\zeta\,(\Gamma^*)^2\left(1 - \frac{\varphi_Z \alpha \Gamma^*}{1+\alpha\Gamma^*}\right), \qquad c(r) = \mathrm{EN}(r)\,e^{\Gamma(r)+B(r)} - \Gamma(r) - 1, \tag{51}$$

with $(\zeta, \varphi_Z, \alpha)$ fixed by requiring, simultaneously:

(A) the first zero-separation theorem, $B(0) = \beta\mu_{\mathrm{ex}} - \Gamma(0)$, with $\beta\mu_{\mathrm{ex}}$ and $\Gamma(0)$ both closed-form from Carnahan–Starling (the latter needing one correction integral $I_C$, evaluated numerically here directly from the current solve's own $g(r)$ and $c(r)$, rather than the original paper's own reliance on an external correlation for it);
(B) the second zero-separation theorem, $dB/d\Gamma^*|_0 = 1/y(\sigma^+) - 1$ – **sign-corrected**: the original 1995 paper's own Eq. (3.8) states the opposite sign; re-deriving this directly from $B(r) = \ln y(r) - \Gamma(r)$ independently confirms the later 1999 erratum's [25] sign instead, which is what is implemented here;
(C) pressure consistency (compressibility route = virial route), the same condition Rogers–Young-family closures use (Section 4's own `findThermodynamicallyConsistentParameter()`), reused here in a three-parameter-aware form.

Matching the validated practice of Pihlajamaa & Janssen (2024) [2] – who report being unable to robustly satisfy all three conditions via exact simultaneous root-finding – this port minimises the sum of squared residuals (`scipy.optimize.least_squares`) rather than attempting an exact $3 \times 3$ nonlinear solve.

**Hard spheres only, enforced twice.** ZSEP's three conditions are derived specifically from the exact Carnahan–Starling hard-sphere equation of state; they are not meaningful for any other potential. This is checked in two places, not one: `OZsolver.fitZSEPparameters()` itself refuses (prints a warning, returns `None` without solving) if the active potential is not `HardSphere`, and – added specifically because that alone would otherwise let `ozLib.solve()` silently package up an all-zeros/$S(Q)$=1-everywhere result with no exception, only an easily- missed Log-tab message – `ozLib.solve()` now raises `ValueError` immediately if `closure='ZSEP'` is requested with any potential other than `'HardSphere'`, before touching the solver at all.

**Known, documented ill-conditioning (not a bug).** This 3-parameter fit is genuinely underdetermined: a whole family of $(\zeta, \varphi_Z, \alpha)$ triples can satisfy the three conditions equally well, and an unconstrained fit can land on a mathematically valid but unphysical-looking point (e.g. very large $\varphi_Z$, $\alpha$ near 0) far from Lee's own reported values – matching Fernaud, Lomba & Lee (2000)'s [23] own explicit finding of multiple solutions to the same optimisation. Fixing $\alpha = 1.0$ (`fitZSEPparameters(fixedParam=('alpha', 1.0))`, Lee's own empirical finding, Table II of [24]: $\alpha \approx 1.0$ works well at every density from 0.1 to 0.9) and starting near literature values gives good, well-behaved results in practice; fully unconstrained 3-parameter fits should have their output sanity-checked (all three parameters within a physically reasonable $\mathcal{O}(0.1)$–$\mathcal{O}(10)$ range) before being trusted.

### 4.21 Euler–Rahman (EuRah) — not offered

Based on Eu & Rah, *J. Chem. Phys.* **111**, 3327 (1999) [1], "A closure for the Ornstein–Zernike relation that gives rise to the thermodynamic consistency." Unlike every closure above, $c(r)$ here is not a local function of $\Gamma(r)$ alone: it is fixed externally to an array $c_{\mathrm{EuRah}}(r)$ that must itself satisfy an outer self-consistency condition,

$$c_{\mathrm{EuRah}}(r) = \frac{1}{36}\Big[\mathrm{MAYER}(r)\Big]\left[36\,y(r) + 18\varphi\frac{\partial y}{\partial r} + 12\,r\frac{\partial y}{\partial \varphi} + 6\varphi\,r\frac{\partial^2 y}{\partial r\,\partial\varphi}\right], \tag{52}$$

where $\mathrm{MAYER}(r) = \mathrm{EN}(r) - 1$, $y(r)$ is the cavity function $y(r) = g(r)e^{U(r)/k_BT}$ evaluated at the trial $c_{\mathrm{EuRah}}$, and the density derivatives are evaluated by re-solving the OZ equation at $\varphi \pm 0.01\varphi$ for the same trial $c_{\mathrm{EuRah}}$. Structurally, writing $36y(r)$ as the dominant term, Eq. 52 is exactly "PY closure plus systematic density/spatial-derivative correction terms" ($c_{\mathrm{PY}} = \mathrm{MAYER} \cdot y$), which is the expected shape for a closure designed to restore thermodynamic consistency to PY.

*This closure is implemented (`OZsolver.solveEuRah()`, `doEuRahClosure()`) but is deliberately excluded from `ozLib.CLOSURE_SETTERS` and therefore from both the GUI dropdown and `ozLib.solve()`'s validated closure list.* Extensive testing found a genuine exponential positive-feedback instability intrinsic to Eq. 52's own structure: inside the hard core, $y(r) = e^{\Gamma(r)}$, and $\Gamma(r)$ itself depends on $c_{\mathrm{EuRah}}(r)$ through the OZ equation, so a more negative trial $c_{\mathrm{EuRah}}$ produces an exponentially larger $y(r)$, which (via the dominant $36y(r)$ term) produces an even more negative $c_{\mathrm{EuRah}}$ on the next iteration. This defeated every numerical strategy attempted: Newton–Krylov (`scipy.optimize.newton_krylov`), damped fixed-point iteration (multiple damping factors), density continuation with warm restarts, dense-Jacobian Newton methods (`fsolve`/`hybr`, Levenberg–Marquardt, Broyden1/2, `df-sane`), a trust-region-style adaptive step limiter, and SUNDIALS' own KINSOL (via `sundials4py`) with this project's best-validated tuning parameters (Section 6.5) — the last of these even produced low-level crashes (segfault/bus error) rather than merely failing to converge. The formula's overall structure was independently verified as plausible (see above), and the literature explicitly notes that this class of "second-order" closure is excluded from standard closure-comparison studies for "numerical complexity" [2]; the instability found here is consistent with that being a genuine property of the theory rather than an implementation error.

### 4.22 Practical guidance: which closure for which potential

Choosing a closure is a tradeoff between accuracy, generality, and cost (RHNC/RMSA need a reference sub-solve; RY/HMSA/MHNC/BPGG/CJVM/BB need either a manually-chosen parameter or an expensive `findThermodynamicallyConsistentParameter()` search; ZSEP self-fits three parameters by nonlinear least squares). The guidance below reflects standard liquid-state-theory practice together with what has actually been validated in this project (Sections 9, 5) – it is a starting point for exploration, not a guarantee that any one closure is quantitatively correct for a given system without checking against an independent reference (Section 5) where one is available.

- **Hard spheres, or any purely repulsive, short-ranged potential** (HardSphere, SoftSphere, GGCM-n, PenetrableSphere): start with **PY**. It is simple, cheap, and (Section 9's own validation) matches the exact hard-sphere solution essentially exactly at the densities tested here – though see Section 4.23 below: against genuine Monte Carlo data (not the exact PY solution itself), PY's own contact-value error is not actually small at high density [2]. **ZSEP** (Section 4.20) is a genuinely more accurate option specifically for HardSphere (only), if its known ill-conditioning (fix $\alpha = 1$, check the fitted parameters look physically reasonable) is acceptable for the use case.
- **Attractive or narrow-well potentials** (StickyHardSphere, SquareWell, LennardJones, PiecewiseConstant): PY tends to be least accurate here (Section 9's own closure comparison shows PY and HNC bracketing the true contact value from opposite sides at high density, a general feature that sharpens with attraction, confirmed quantitatively against Monte Carlo in Section 4.23 [2]); an empirical bridge-function closure (**Verlet**, **MS**, **VM**, **DH**, **CG**) or **HNC** itself is generally a better default. **HMSA** (needs a manually chosen or fitted $\alpha$) is specifically designed for exactly this case – a potential with a genuine attractive tail split from a repulsive core – since it uses that split directly (Section 4's own HMSA formula). For narrow/strong wells specifically, cross-check against the semi-analytical references built for exactly this case (Sharma–Sharma, ORPA, RFA, Section 5)

if available for the potential in use, since this project's own worked examples show a naive perturbative treatment (Sharma–Sharma) can degrade substantially for strong/wide wells where a genuinely non-perturbative one (ORPA) does not. For **StickyHardSphere** specifically, published Monte Carlo comparisons [26] find plain PY performs adequately in the parameter regime typical of colloidal/protein solutions, but is out-performed by more sophisticated closures elsewhere – see Section 4.23.

- **Long-range, charged, or screened-Coulomb potentials** (DLVO, DLVO Hydra, HS3Yukawa, IonicMicrogel): **MSA** is the standard, cheap starting point, but can give an unphysical negative contact value at strong coupling (Section 9.3); **RMSA** is the direct fix for exactly that failure mode and should be preferred whenever MSA's own contact value comes out negative. Where the closed-form Hayter–Penfold solution applies (a bare hard-core-Yukawa potential, Section 5.4), that semi-analytical reference is both faster and, being the historically canonical solution for this exact problem, a useful independent check on the general numerical solver's own RMSA rescaling. **HNC**, by contrast, is well-documented in the plasma/charged-colloid literature to degrade, and in some regimes fail to converge at all, at strong coupling (Section 4.23) – this is specifically why Rosenfeld's variational modified HNC (VMHNC) [27] and similar corrections exist; **BPGG** (already offered here, and itself one of the corrections used in that literature) is a reasonable substitute for plain HNC at strong coupling if MSA/RMSA is not applicable.
- **When thermodynamic consistency matters most** (equation-of-state or virial-route quantities, not just $g(r)/S(Q)$ shape): **Rogers–Young** (or **HMSA** for potentials with a genuine attractive tail) with `findConsistentParameter=True` is specifically designed to enforce this by construction, at the cost of the extra root-finding search (Section 7). **MHNC**, **BPGG**, **CJVM**, and **BB** offer the same consistency search (`CONSISTENT_PARAMETER_CLOSURES`) as alternative bridge-function forms, worth trying if Rogers–Young/HMSA's own search fails to find a genuine root for a particular system.
- **Highest achievable general accuracy, cost secondary**: **RHNC** (a genuine reference-system correction, no free parameter to choose) or **MHNC** (a fixed analytical PY bridge function, cheaper than RHNC since no reference sub-solve is needed) are the more sophisticated options among what is offered here – consistent with Section 4.23's own finding that MHNC-type closures (parameterised directly from simulation data) systematically outperform parameter-free closures against genuine Monte Carlo benchmarks [2].
- **Avoid**: **Euler–Rahman** (Section 4.21) – excluded from the GUI/`ozLib.solve()` entirely for a genuine, documented convergence instability, not merely a missing feature. The hand-written **Anderson acceleration** solver (Section 6, not a closure but worth flagging here too) should not be trusted uncross-checked on strongly-coupled/charged systems specifically, having been found to converge silently to a wrong answer there in this project's own testing (Section 9.4).
- **Kovalenko–Hirata**: now implements the real literature formula (Section 4.17), not the SASfit-C-code quirk an earlier version of this port had (which was, in effect, plain HNC under a different name). Its main practical advantage over plain HNC is bounded, non-exponential behaviour when $\Gamma(r)$ grows large, per the literature [28] – worth trying wherever HNC itself is a reasonable choice (Section 4.23) but its own exponential growth is a concern.

### 4.23 Literature validation against Monte Carlo: known accurate and inaccurate combinations

The recommendations above draw on published, quantitative comparisons against Monte Carlo (MC) or molecular dynamics (MD) simulation – the genuine ground truth for a given potential, as opposed to a closure's own internal thermodynamic consistency, which (Section 4's own introduction) can be satisfied without the correlation functions themselves being accurate. This

section summarises what is actually documented in the literature for the specific potential/closure pairs offered here, including where combinations are *known to give poor results*, not only where they succeed – since a plugin's own thermodynamic self-consistency check passing is not, by itself, evidence of accuracy against MC.

**Hard spheres.** Pihlajamaa & Janssen (2024) [2] performed a systematic MC comparison across many closures at $\rho\sigma^3 = 0.94$ (just below melting) and report the PY and HNC closures as the two *worst*-performing of every closure they tested, PY underestimating and HNC overestimating the contact value: $g_{\mathrm{PY}}(\sigma) - g_{\mathrm{MC}}(\sigma) = -0.921$, $g_{\mathrm{HNC}}(\sigma) - g_{\mathrm{MC}}(\sigma) = +1.966$ – confirming the long-known qualitative result that PY outperforms HNC for hard spheres, but quantifying just how large both errors are at this density. Across every system they tested (hard spheres, inverse-power-law, Gaussian-core, and Lennard-Jones, in two and three dimensions), their overall conclusion is that "the well-known Percus-Yevick equation performed worse than any among the V[erlet], DH, and MS closures" – i.e. PY's continued popularity reflects familiarity and ease of implementation, not superior accuracy, a point their paper makes explicitly. The Lee/ZSEP closure specifically (Section 4.20) was found to give a *better* contact value than the other thermodynamically-consistent closures they tested, which the authors attribute directly to its built-in use of the Carnahan–Starling equation of state – independent confirmation, from outside this project, of exactly the ZSEP behaviour already documented in Section 4.20.

**Sticky/adhesive hard spheres.** Gazzillo & Giacometti (2004) [26] solved Baxter's sticky-hard-sphere model analytically for several closures (including PY) and compared directly against MC. Their finding is a genuine middle-of-the-road result for PY, not a clear pass or fail: PY's predictions "lie between simpler closures, which may yield less accurate predictions but are easily extensible to multicomponent fluids, and more sophisticat[ed] closures which give more precise predictions but can hardly be extended to mixtures." They further report that "in regimes typical for colloidal and protein solutions," first-order perturbative closures (including PY) "produce satisfactory results" – i.e. PY's adequacy for sticky spheres is regime-dependent, not universal, consistent with this project's own choice to offer the Baxter/PY solution (Section 5) as a validated reference specifically for that regime rather than as a claim of general accuracy.

**Lennard-Jones.** The same Pihlajamaa & Janssen study [2] finds that renormalising the indirect correlation function ($\gamma^*(r) = \gamma(r) - \beta u_{\mathrm{LR}}(r)$, the same attractive/repulsive potential split this project's own HMSA/RY closures use, Section 3) is necessary for good accuracy: plain PY applied without this renormalisation is markedly less accurate than the renormalised closures (CG*, Verlet*, DH*) at their tested state point ($\rho\sigma^3 = 0.8$, $k_BT/\varepsilon = 1.0$, peak $g(r_{\mathrm{peak}}) \approx 2.6$).

**Charged and screened-Coulomb (Yukawa) systems.** Unlike the short-ranged cases above, the literature on strongly-coupled Yukawa/one component-plasma systems documents outright *failure* of plain HNC, not merely reduced accuracy: HNC is known not to have a solution at all in certain regions of the phase diagram approaching phase separation for hard-core-Yukawa and related charged systems, with the boundary of non-existence not coinciding with the true spinodal. This is the direct motivation for Rosenfeld's variational modified HNC (VMHNC) [27] and later isomorph-based modified-HNC variants, both explicitly constructed and validated against MD specifically because plain HNC's accuracy (and, in some regimes, its very convergence) degrades as coupling strength increases for these systems – directly analogous to, and part of the same broader literature as, the MSA contact-value failure this project's own RMSA rescaling addresses (Section 4.6) for the closure actually offered here. **BPGG** (Section 4), one of the closures used in that same strongly-coupled-Yukawa literature as an HNC alternative, is offered directly in this toolkit and is a reasonable next step if RMSA is not applicable to the potential at hand.

*Scope of what is, and is not, independently confirmed here:* the findings above are drawn directly

from the cited papers' own stated results, not re-derived or re-run by this project. Where a cited paper studies a closely related but not identical system (e.g. the one-component-plasma/Yukawa HNC-convergence literature, which concerns bulk charged fluids rather than this project's own colloidal DLVO/HS3Yukawa potentials specifically), this is noted as such above rather than presented as a direct one-to-one validation of this project's own numerical results.

# 5 Analytical and Semi-Analytical Cross-Validation Tools

Independent of the general numerical OZ solve, several closed-form or semi-analytical structure-factor solutions are implemented as cross-checks. These are consulted throughout this project to validate the numerical solvers and closures above, rather than offered as alternative closures themselves.

## 5.1 Percus–Yevick hard sphere (Wertheim–Thiele)

The classic closed-form PY hard-sphere solution (Naegele's notation),

$$S(q) = \frac{1}{X(q)^2 + Y(q)^2}, \quad X = 1 - 12\varphi(Af_1 + Bf_2), \quad Y = -12\varphi(Af_3 + Bf_4), \tag{53}$$

with $A = (1 + 2\varphi)/(1 - \varphi)^2$, $B = (1 + \varphi/2)/(1 - \varphi)^2$, and $f_1, \ldots, f_4$ standard trigonometric combinations of $y = q\sigma$. Verified directly against SASfit's own `GPY()` formula; found the existing Python implementation to be numerically *more* robust than SASfit's own C code at very low $q$ (SASfit's small-argument Taylor threshold is far tighter than where the raw trigonometric formula actually loses precision).

## 5.2 Baxter sticky hard sphere (idealised and finite-well)

Baxter's (1968) closed-form adhesive-sphere solution using stickiness parameter $\tau$, ported directly from `sasfit_sq_sticky_hard_sphere.c`, plus a finite-well generalisation (`sasfit_sq_sticky_hard_spher` for a well of finite width $\delta$ — the physically correct comparison for this project's own finite-$\delta$ `StickyHardSphere` potential (Section 3.17), since the idealised $\delta \to 0$ limit is not the same physical system.

## 5.3 Sharma–Sharma square well

The classic first-order perturbative PY square-well solution (Sharma & Sharma, *Physica A* **89**, 213 (1977)), ported from `sasfit_sq_square_well_potential.c`:

$$S(q) = \frac{1}{1 - C(q)}, \qquad C(q) = \frac{-24\varphi}{\kappa^6}\Big[\alpha\,\kappa^3(\sin\kappa - \kappa\cos\kappa) + \beta\,\kappa^2\big(2\kappa\sin\kappa - (\kappa^2 - 2)\cos\kappa - 2\big) \tag{54}$$

$$+\gamma\big((4\kappa^3 - 24\kappa)\sin\kappa - (\kappa^4 - 12\kappa^2 + 24)\cos\kappa + 24\big) - \varepsilon\,\kappa^3(\sin\lambda\kappa - \lambda\kappa\cos\lambda\kappa + \kappa\cos\kappa - \sin\kappa)\Big], \tag{55}$$

with $\kappa = q\sigma$, well outer range $\lambda = 1 + \delta/\sigma$, and $\alpha, \beta, \gamma$ the same PY-like density coefficients as the hard-sphere solution above. Being a first-order expansion, this closed-form solution degrades for strong/wide wells (verified: max. deviation from the full numerical solve rose from $\sim 0.005$ at $\varepsilon = -0.02, \delta = 0.02\sigma$ to $\sim 0.33$ at $\varepsilon = -0.5, \delta = 0.3\sigma$, at $\varphi = 0.15$), which is the expected behaviour of a perturbative expansion approaching the edge of its own validity.

## 5.4 Rescaled MSA (Hayter–Penfold)

The full Hayter–Penfold RMSA solution for a hard-core Yukawa/screened-Coulomb potential, compiled as a standalone C library (`rmsa_c_source/`, from `src/plugins/RMSA/` – `rmsa.c`, `polyroots.c`, `rmsa_physical.c`) and wrapped via `ctypes` (`rmsaWrapper.py`), reusing SASfit's own quartic-root selection and small-$\kappa$ Taylor-series logic rather than re-implementing it. Validated: the weak-coupling limit reproduces the exact PY hard-sphere contact value $g(\sigma^+) = (1 + \varphi/2)/(1 - \varphi)^2$.

### 5.5 One-Yukawa MSA (Blum–Høye)

A semi-analytical Yukawa-MSA solution, compiled as a standalone C library (`oneyukawa_c_source/`, from `src/plugins/twoyukawa/`) including a full Jenkins–Traub complex polynomial root-finder (`2Y_cpoly.c`) and an FFT-based pair-correlation-function evaluation (`2Y_PairCorrelation.c`) used to select the physically correct root among up to four candidates. Cross-validated against the general numerical OZ solve (`HS3Yukawa` potential + MSA closure) to within $\sim 2\%$ mean / $\sim 5\%$ max deviation, concentrated at low $q$ and shrinking rapidly at higher $q$ — the same order of residual found for the exact PY hard-sphere case, consistent with a numerical-grid-vs-exact discrepancy rather than an error in either implementation.

### 5.6 ORPA (Pini–Parola–Reatto) for square wells

A self-consistent optimised random-phase approximation specific to square-well potentials (Pini, Parola & Reatto, *Mol. Phys.* **100**, 1507 (2002)), compiled as a standalone C library (`orpa_c_source/`) using GSL's derivative-free multivariate root solver with continuation in the well depth. At a strong-well test case ($\varphi = 0.15$, $\varepsilon = -0.5 k_B T$, $\delta = 0.3\sigma$) where the Sharma–Sharma perturbative formula (Section 5.3) showed max. deviation 0.33 from the numerical solve, ORPA gave max. deviation 0.057 — roughly $6\times$ closer, consistent with ORPA being a genuinely non-perturbative theory.

### 5.7 RFA (Rational Function Approximation)

Santos, Yuste & López de Haro's rational-function approximation for *arbitrary* piecewise-constant potentials (*Condens. Matter Phys.* **15**, 23602 (2012)), compiled as a standalone C library (`rfa_c_source/`) with a numerically-safe small-$q$ series expansion. Because it handles an arbitrary number of piecewise-constant steps, this is the only analytical tool validated against both the single-well `SquareWell` potential ($n = 1$; max. deviation 0.025 on the same strong-well case above, the closest of the three square-well cross-checks) *and* the genuinely multi-step `PiecewiseConstantHS` potential ($n = 3$ shoulder-well-shoulder; max. deviation 0.0056/mean 0.0028), a cross-check no other tool here can perform.

# 6 Numerical Solver Algorithms

Given a closure (Section 4), the OZ equation (Eq. 2) reduces the physics problem to a fixed-point problem $\Gamma = \mathcal{F}(\Gamma)$, where $\mathcal{F}$ composes the closure and the Hankel-transform-based OZ update. Eight solver algorithms are available, offering a range of speed/reliability tradeoffs (Section 6.7 compares them empirically).

## 6.1 Picard iteration

Plain, undamped fixed-point iteration, $\Gamma_{n+1} = \mathcal{F}(\Gamma_n)$, until the relative change in $\|\Gamma_n\|$ falls below a convergence criterion. Simple and reliable for easy problems; can fail to converge for harder ones (e.g. dense hard spheres, strongly-charged systems).

## 6.2 Anderson acceleration (hand-written)

A direct Python implementation of Anderson mixing: the next iterate is a linear combination of the last $m$ residuals, with mixing coefficients chosen by a small least-squares problem, extrapolating the fixed-point iteration rather than taking it verbatim. Confirmed independently reliable in most cases, but found (Section 6.7) to occasionally converge to a spurious answer on very hard problems (strongly-charged MSA) without raising any error — silently disagreeing with cross-validated Newton–Krylov solutions by a wide margin in that regime.

## 6.3 scipy Anderson / scipy Newton–Krylov

Thin wrappers around `scipy.optimize.anderson/newton_krylov` respectively, applied to $\mathcal{F}$. Reliable across the full range of problems tested here, though can be markedly slower than the alternatives below on both easy and hard problems.

## 6.4 Biggs–Andrews

A hand-written acceleration scheme, ported directly from `sasfit_oz_solver.c`'s own `case BIGGS_ANDREWS:` block (dispatched there through the same `OZ_solver_by_iteration()` as Picard — i.e. this is *not* SUNDIALS-based, despite the KINSOL-adjacent naming of its own tuning parameter). After two unaccelerated warm-up steps, each further step:

1. computes a Barzilai–Borwein-style spectral step size $\alpha = \beta/\gamma$ (or $\mathrm{sgn}(\beta/\gamma)\sqrt{|\beta/\gamma|}$), with $\beta = \langle g_n, g_{n-1}\rangle$, $\gamma = \langle g_{n-1}, g_{n-1}\rangle$ inner products of consecutive fixed-point residuals $g_k = \mathcal{F}(x_k) - x_k$;
2. clamps $\alpha$ to the Nesterov momentum bound $\left[-\frac{n-1}{n+2}, \frac{n-1}{n+2}\right]$;
3. extrapolates using order $|$`KINSetMAA`$|$ (0: none; 1: heavy-ball momentum $y = x + \alpha(x - x_{\mathrm{prev}})$; $\geq$2: adds a second-order term $\frac{\alpha^2}{2}(x - 2x_{\mathrm{prev}} + x_{\mathrm{prevprev}})$).

Validated: order 0 reproduces plain Picard almost exactly (354 vs. 353 steps on hard-sphere/PY at $\varphi = 0.3$); orders 1/−2 roughly halve the iteration count on that same case; every configuration agrees with the independently validated GMRES solver to $\sim 10^{-12}$.

## 6.5 SUNDIALS KINSOL: Newton–Krylov (GMRES/FGMRES/TFQMR)

A genuine Newton–Krylov method via SUNDIALS' own KINSOL, through its official Python bindings (`sundials4py`), using the `KIN_LINESEARCH` strategy with a selectable linear solver. Tuning parameters were matched exactly to SASfit's own "configure OZ solver" dialog defaults (shared directly by a user of this project): `KINSetMaxNewtonStep`$= 100N$ (not just $N$),

`KINSetFuncNormTol`= $10^{-10}$, `KINSetScaledStepTol`= $10^{-13}$, `KINSetMaxRestarts`= 10 (GMRES/FGMRES only), `KINSetEtaForm`= `KIN_ETACHOICE1`. This made a confirmed, material difference: a strongly-charged DLVO+MSA case that failed to converge under more conservative defaults converges cleanly under these settings. `SUNLinSol_SPBCGS` (Bi-CGSTAB) is deliberately not offered: it segfaults unconditionally in the tested `sundials4py` version, confirmed in total isolation from any OZ-specific code, i.e. a bug in that package rather than a usage error here.

### 6.6 SUNDIALS KINSOL: Fixed-Point (Anderson), `KIN_FP`

SUNDIALS' own Anderson-accelerated fixed-point strategy. This needed a genuinely different `sysfn` convention than the Newton–Krylov solver above — found, by reading `OZ_step_kinsolFP()` in the C source directly, that `sysfn` must return $\mathcal{F}(x)$ itself, not the residual $\mathcal{F}(x)-x$ every other KINSOL-based solver here uses. Once matched, no Anderson damping (`KINSetDampingAA`) was needed, matching SASfit's own configuration function exactly (which never sets it either); validated to $10^{-12}$ against the Newton–Krylov solver on two different potentials.

### 6.7 Empirical solver comparison

| Case | Picard | Anderson | scipy NK | Biggs–Andrews | sundials4py GMRES/KIN_FP |
|---|---|---|---|---|---|
| Easy (HS+PY) | OK | OK (fastest) | OK | OK | OK (fastest) |
| Moderate (Sticky+MSA) | OK | OK | OK | OK | OK |
| Dense HS ($\varphi$=0.45) | **fail** | OK | OK | OK (order$\geq$1 only) | OK |
| Strongly-charged MSA | **fail** | *wrong answer, uncaught* | OK | **fail** | GMRES: OK; KIN_F |

Table 1: Reliability across a range of difficulties (see the project history for full timing data). No single solver dominates; `sundials4py` GMRES/FGMRES was the only combination to succeed on every case tried. The hand-written Anderson solver's silent wrong-answer failure mode on the hardest case is a more concerning outcome than an honest non-convergence report.

# 7 The ozLib Python Library

ozLib.py is the single source of truth for the calculation workflow — both oZgui.py's own "calculate" button and any user script call the same ozLib.solve() function, so results from the GUI and from scripts are identical by construction, not by two independently-maintained implementations happening to agree.

## 7.1 Basic usage

```
from ozLib import solve

result = solve(potential='HardSphere', phi=0.3,
               closure='Percus-Yevick',
               solver='sundials4py: Newton-Krylov (GMRES)')

result.gr # g(r)
result.Sq # S(q)
result.cr # c(r)
result.gamma # Gamma(r)
result.hr # h(r)
result.Ur # U(r)/kT
result.Br # B(r) (bridge function)
result.yr # y(r) (cavity function)
result.fr # f(r) (Mayer function)
result.r, result.q # the real-/reciprocal-space grids
```

## 7.2 Function signature

```
def solve(potential, phi, potentialArgs=(), closure="Percus-Yevick",
          closureParam=None, findConsistentParameter=False,
          solver=_defaultSolverName, maxIterations=1000,
          numberOfRadialSamplingPoints=None, hardSphereDiameterInPoints=None,
          onSolverCreated=None):
    ...
    return OZResult(...)
```

potential, closure, and solver are validated against ozLib.SOLVER_CLASSES/ozLib.CLOSURE_SETTERS (the canonical, current option lists — print them from a script to see exactly what is available); an unrecognised name, or a closure needing closureParam without one given, raises ValueError immediately rather than failing confusingly deeper inside the solve. numberOfRadialSamplingPoints/hardSphereDiameterInPoints expose the grid configurability of Section 2 directly. solver's default, _defaultSolverName, is *not* "Picard iteration" (an earlier version of this document, and an earlier version of the code itself, both had it as the default) — it is now "sundials4py: Fixed-Point (Anderson)" when the optional sundials4py dependency is installed, falling back to "scipy Anderson" otherwise; see ozLib.py's own comment on SOLVER_CLASSES for why (SUNDIALS' production-grade Anderson-accelerated fixed-point strategy generally converges more reliably and faster than plain unaccelerated Picard for the harder closures — Section 6.7 quantifies this). Plain Picard iteration remains fully available (solver="Picard iteration"), simply no longer pre-selected. findConsistentParameter: for closures with a free parameter that also have a published thermodynamic-consistency condition determining it automatically (Rogers–Young, HMSA, Modified HNC, BPGG, CJVM, BB — see ozLib.CONSISTENT_PARAMETER_CLOSURES), setting this to True searches for that value instead of requiring closureParam from the caller, via OZsolver.findThermodynamicallyConsistentParameter() (Rogers & Young's own compressibility-route/virial-route consistency idea, generalised across potentials rather than limited to any one). Raises ValueError if requested for a closure that doesn't support it.

### 7.3 Example: potential with arguments and a parametrised closure

```
result = solve(potential='StickyHardSphere', phi=0.2,
               potentialArgs=(0.3, 0.05), # (tau, delta)
               closure='Rogers-Young', closureParam=2.0,
               solver='scipy Newton-Krylov')
```

### 7.4 Example: long-range potential, extended grid

```
result = solve(potential='HS3Yukawa', phi=0.15,
               potentialArgs=(-0.3, 10.0, 0, 1, 0, 1), # (K1, lambda1, ...)
               closure='MSA',
               numberOfRadialSamplingPoints=4*4096, # 4x extended range
               hardSphereDiameterInPoints=100) # same resolution
```

## 8 The ozGUI Application

`oZgui.py` is a Tkinter application, run directly with `python oZgui.py`, providing:

- **Left panel:** potential dropdown (with dynamically-generated parameter fields, discovered by introspection — a new potential added to `oZfixpointOperator.py` appears automatically, nothing else to edit), closure dropdown (with a parameter field where needed, plus a "find thermodynamically consistent value" checkbox for closures with a published consistency condition, see `CONSISTENT_PARAMETER_CLOSURES` in Section 7, which disables — rather than hides — the manual entry field while checked), volume fraction, solver dropdown, X-axis scale selector (`symlog`/`linear`/`log`/`asinh`, with an adjustable linear-region threshold), grid size controls (`Grid points (N)`/ `Points per` $\sigma$, both defaulting to `"auto"` to keep this project's own historical grid, Section 2; either can be overridden independently, e.g. to extend the range for a long-range potential without losing near-contact resolution), max. iterations, calculate/interrupt/clear/delete-last buttons, Save/Load (`.oz` JSON format, round-trips exactly including NaN entries in $B(r)/y(r)$), and a single format-selector export control (ASCII/CSV/Excel/PNG-all-tabs/copy- to-clipboard) for the currently selected (or most recent) run.
- **Right panel:** a 10-tab notebook — the same 9 plot tabs as the original Tcl GUI (S(Q), g(r), c(r), $\Gamma(r)$, h(r), U(r)/kT, B(r), y(r), f(r)), overlaying every run in the history list in a distinct colour, plus a **Log tab** with no Tcl-side equivalent: every status update and every `print()` anywhere in the whole process (including diagnostic output already buried inside `oZfixpointOperator.py`/`oZsolver.py`, e.g. `fitZSEPparameters()`'s or `findThermodynamicallyConsistentParameter()`'s own messages) is captured here via a thread-safe `stdout`/`stderr` redirector, timestamped, and tee'd to the real console so nothing is lost even when run windowed (no terminal at all).

Exported ASCII/CSV/Excel data always contains the full-precision computed values regardless of the current plot's axis scale (verified directly: two exports taken with different X-axis scale/threshold settings in between are byte-identical); only the PNG export reflects the currently displayed scale, since it is a rendered image.

### 8.1 Relationship to SASfit's own Tcl GUI, and naming-convention differences

`oZgui.py` is a direct, from-scratch port of SASfit's own built-in Ornstein–Zernike solver window, `sasfit.vfs/lib/app-sasfit/tcl/sasfit_OZ_solver.tcl` — the same potential/closure/algorithm choices, the same 9 output curves, laid out as a Tkinter equivalent of that Tcl/Tk/BLT window rather than embedded inside SASfit itself. Three things were deliberately *not* carried

over, since they either depend on SASfit's own fitting engine directly or are already covered by a Tkinter/matplotlib equivalent: the "assign to SQ plugin" mechanism (replaced by the Save/Load/export controls of Section 8 instead), the detailed KINSOL tuning sub-dialog (see Table 5 below), and crosshair coordinate readout / zoom-stack (matplotlib's own toolbar under each plot covers zoom/pan/save already).

Cross-referencing the Tcl source directly against `oZgui.py`/ `ozLib.py` (rather than assuming a 1:1 naming correspondence) surfaced a number of places where the same concept is named differently on the two sides, summarised in Tables 2–6 below. The general pattern: Tcl consistently favours short abbreviations matching the underlying SUNDIALS/mathematical-literature names exactly (`RMSA`, `mMSA`, `MS`, `DH`, `KINSetMAA`); this Python port consistently favours spelled-out, more self-explanatory names for anything user-facing (`Rescaled MSA`, `Modified MSA`, `Martynov-Sarkisov`, `Duh-Haymet`), while keeping terse internal dict keys (`gr`, `cr`, `Sq`, ...) at the data-storage layer — i.e. the renaming happens specifically at the human-facing label layer, not throughout.

| Tcl `OZ(potential)` value | Python potential name |
|---|---|
| `HardSphere`, `StickyHardSphere`, `SquareWell`, | same |
| `PiecewiseConstant`, `SoftSphere`, `Fermi`, | same |
| `LennardJones`, `IonicMicrogel`, `PenetrableSphere`, | same |
| `DLVO`, `DLVO Hydra`, `GGCM-n`, `HS 3Yukawa` | same |
| `StarPolymer (f>10)` | `StarPolymerHighF` (renamed, not identical) |
| `StarPolymer (f<10)` | `StarPolymerLowF` (renamed, not identical – previously missing entirely, Section 3.4) |
| `Depl-Sph-Sph` | |
| `Depl-Sph-Discs` | *not currently offered* |
| `Depl-Sph-Rods` | (Section 3.5's Known gap) |

Table 2: Potential-name naming convention. Most match exactly; star polymer now matches in *substance* (both formulas present) but not exact spelling, since Python identifiers can't contain spaces/parentheses/ inequality signs; the three depletion geometry variants remain a known, unfixed gap. Python's own list is generated dynamically via introspection of `oZfixpointOperator.py`'s `set<Name>Potential()` methods, so it automatically includes any future addition there; Tcl's is a fixed, hand-written `-values {...}` string that must be edited by hand.

*Not independently re-verified in this pass:* the exact Python argument names of `oZfixpointOperator.py`'s `set<Name>Potential()` methods were not individually confirmed character-for-character against the Tcl GUI's own hand-written parameter display labels for every one of the 18 potentials — most match directly (e.g. `diameter`, `tau`, `delta`, `epsilon`), but three potentials (`SoftSphere`'s "stiffness n", and the two `Depl-Sph-*` potentials' "diam. (large)"/"diam. (small)"/"n (numb. dens.)" style labels) have Tcl-side display strings that are abbreviated prose rather than bare identifiers, making them the most likely spots for the actual Python argument's spelling to differ from what the Tcl label alone would suggest.

# 9 Worked Examples: Numerical vs. Analytical Validation

This section collects representative comparisons between the numerical solver and the analytical/semi-analytical references of Section 5, plus the empirical behaviour of the toolkit at difficult and physically interesting parameter combinations.

| Tcl `OZ(closure)` value | Python closure name | Relationship |
|---|---|---|
| `Percus-Yevick` | `Percus-Yevick` | identical |
| `Hypernetted-Chain` | `Hypernetted-Chain` | identical |
| `Reference HNC` | `Reference HNC` | identical |
| `modified HNC` | `Modified HNC` | capitalisation only |
| `PLHNC` | — | Tcl only |
| `MSA` | `MSA` | identical |
| `RMSA` | `Rescaled MSA` | **renamed** (abbrev. vs. spelled out) |
| `mMSA` | `Modified MSA` | **renamed** |
| `SMSA` | `Symmetric MSA` | **renamed** |
| `HMSA` | `HMSA` | identical |
| `Rogers-Young` | `Rogers-Young` | identical |
| `Verlet` | `Verlet` | identical |
| `MS` | `Martynov-Sarkisov` | **renamed** (initials vs. full) |
| `DH` | `Duh-Haymet` | **renamed** (initials vs. full) |
| `Vompe-Martynov` | `Vompe-Martynov` | identical |
| `BB`, `BPGG`, `CJVM` | `BB`, `BPGG`, `CJVM` | identical (kept as initials both sides) |
| `Choudhury-Gosh` | `Choudhury-Ghosh` | **spelling differs** (Python matches the standard transliteration) |
| — | `Kovalenko-Hirata` | Python only (added after checking SASfit's own `sasfit_oz.h` enum directly) |
| — | `ZSEP` | Python only (Lee 1995 hard-sphere closure) |
| Euler-Rahman/EuRah: excluded from *both* GUIs' dropdowns (Section 4.21) | | |

Table 3: Closure-name naming convention: the largest divergence of any category compared here.

## 9.1 Hard-sphere Percus–Yevick: full validation across $\varphi$

Figure 1 compares the numerical solve (`sundials4py` GMRES, Section 6) against the closed-form PY solution (Section 5) across four volume fractions. The circles (analytical) sit exactly on the numerical curves at every $\varphi$ shown, including the characteristic contact-value growth and the shift of the first $S(q)$ peak to higher $q$ with increasing density.

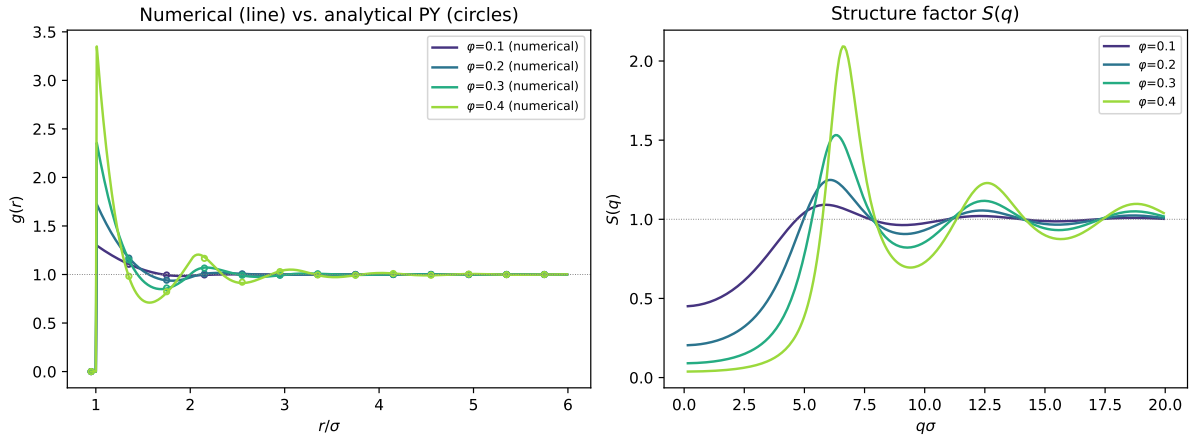


Figure 1: $g(r)$ (left) and $S(q)$ (right) for hard spheres at $\varphi \in \{0.1, 0.2, 0.3, 0.4\}$: numerical solve (solid lines) vs. the exact PY solution (open circles, sub-sampled for visibility).

## 9.2 Closure comparison at high density

Figure 2 shows $g(r)$ for four different closures applied to the same hard-sphere system at $\varphi = 0.4$ (chosen because closure differences are most visible near the freezing transition, where PY's own known contact-value error becomes appreciable). PY and HNC bracket the contact peak from below/above respectively (a well-known, textbook-level result); Verlet and Duh–Haymet, both empirical-bridge-function closures, sit between them and agree with each other almost exactly

| Tcl `OZ(algorithm)` value | Closest Python `solver=` name | Relationship |
|---|---|---|
| `Picard iteration` | `Picard iteration` | identical (the one exact match) |
| `Mann/Ishikawa/Noor/SP/S/CR/Picard-S/PMH/Mann II/Krasnoselskij/S*` iteration | — | Tcl only (many fixed-point iteration variants not offered in this Python port) |
| `dNewton` | `scipy Newton-Krylov` (nearest concept) | not a naming match |
| `Biggs_Andrews` | `Biggs-Andrews` | punctuation only (_ vs. -) |
| `Anderson mixing` | `Anderson acceleration` / `scipy Anderson` | **renamed**, and split into two |
| `KINSOL_FP` | `sundials4py: Fixed-Point (Anderson)` | **renamed and restructured** |
| `GMRES/FGMRES/TFQMR` | `sundials4py: Newton-Krylov (GMRES/FGMRES/TFQMR)` | restructured: Python names the Newton-Krylov family + linear solver |
| `Bi-CGStab` | — | Tcl only (SUNLinSol_SPBCGS segfaul unconditionally in tested `sundials4py` |
| — | `scipy Newton-Krylov` | Python only |

Table 4: Solver/algorithm naming: almost no string-level overlap at all, only `Picard iteration` matches verbatim.

| Tcl name (literal SUNDIALS C API name) | Python equivalent |
|---|---|
| `KINSetMAA` | not directly exposed in the GUI (used internally) |
| `KINSetFuncNormTol` | not directly exposed (`self.convergenceCriterion` instead) |
| `KINSetScaledSteptol` | not directly exposed |
| `KINSetNumMaxIters` | `"Max iterations:"` → `maxIterations=` |
| `KINSetPrintLevel` | not directly exposed (routed through the Log tab instead) |
| `KINSetEtaForm` | not directly exposed |
| `KINSpilsSetMaxRestarts` | not directly exposed |
| `KINSolStrategy` | not directly exposed |
| `KINSetMaxNewtonStep` | not directly exposed |

Table 5: KINSOL tuning parameters: the one naming layer where Tcl uses the literal underlying C API names, not a GUI-specific relabeling. Deliberately not mirrored 1:1 in the Python GUI – `oZgui.py`'s own design choice is sensible per-library defaults (Section 6.5) plus the two simpler knobs above, not a full tuning sub-dialog.

at this density.

### 9.3 Charged systems: RMSA and the MSA contact-value problem

Plain MSA (Section 4) can give an unphysical, negative contact value $g(\sigma^+) < 0$ for sufficiently strongly-coupled charged systems — a well-known limitation of MSA itself, not an artifact of this implementation. Figure 3 shows a representative $S(q)$ from the semi-analytical Hayter–Penfold RMSA solver (Section 5.4), which sidesteps this issue by construction via its own closed-form rescaling.

The general-potential numerical solver's own RMSA rescaling (`OZsolver.solveRMSA()`, Section 4.6) implements the same idea (extend the "apparent" hard-core radius outward until the contact value there is non-negative) for arbitrary potentials, not just the hard-core-Yukawa case Hayter–Penfold covers in closed form. This method was found, on close comparison against `sasfit_oz_solver.c` itself, not to be a plain bisection search at all: SASfit's own algorithm is a *grow, then bisect* scheme, where each solve's own full $g(r)$ array is scanned once to see exactly how far the negative region extends (extending the excluded region as far as that single solve's own data justifies, at no extra cost), and only once a solve gives an already-non-negative result does it fall back to bisecting the gap between the historical bounds. An earlier version of this Python port used plain midpoint bisection instead, which was both less efficient and, on inspection, genuinely buggy; it has since been rewritten to match the C algorithm. **Three genuine**

| Physical quantity | Tcl variable(s) | Python internal key |
|---|---|---|
| Structure factor | `OZ(res,s,x/y)`, `OZ(result,q/Sq)` | `Sq` |
| Radial distribution function | `OZ(res,g,x/y)`, `OZ(result,r/gr)` | `gr` |
| Direct correlation function | `OZ(res,c,x/y)`, `OZ(result,cr)` | `cr` |
| Indirect correlation $\Gamma(r)$ | `OZ(res,gamma,x/y)`, `OZ(result,gammar)` | `gamma` |
| Total correlation function | `OZ(res,h,x/y)`, `OZ(result,hr)` | `hr` |
| Interaction potential/$k_BT$ | `OZ(res,u,x/y)`, `OZ(result,Ur)` | `Ur` |
| Bridge function | `OZ(res,B,x/y)`, `OZ(result,Br)` | `Br` |
| Cavity function | `OZ(res,y,x/y)`, `OZ(result,yr)` | `yr` |
| Mayer-$f$ function | `OZ(res,f,x/y)`, `OZ(result,fr)` | `fr` |

Table 6: Output-curve naming: Python's short internal dict keys match Tcl's own `OZ(result,...)` suffixes almost character-for-character (the one exception being `gammar`→`gamma`), even though the *user-visible tab labels* were rewritten independently (e.g. Tcl's "indirect\ncorrelation\nfunct. gamma(r)" vs. Python's plain "Gamma(r)"). This strongly suggests this specific naming layer was deliberately kept close to the original Tcl variable names even where the rest of the port was not.

**bugs were found and fixed** in total – two in this Python port, one in SASfit's own C source:

1. *Stale data on solver non-convergence* (Python port). Several solver classes (e.g. the `sundials4py`-based ones) simply print a message and return early when the underlying Newton–Krylov iteration fails to converge, without updating `self.radialDistributionFunction` – meaning `getRDF()` silently returned data left over from whichever solve last actually succeeded, corrupting the search's own decisions and final result. Fixed with a solver-class-independent, Picard-iteration-based internal solve (`OZsolver._robustInlineSolve()`), warm-started between successive trials (the same density-continuation- with-warm-restarts principle already used for `solveEuRah()`'s own outer iteration).
2. *An off-by-one tautology in the original bisection-based port.* The old code's gate array, `gate4g[:mid+1]=0`, forced `g[mid]` to be exactly zero *by construction* (it lies inside the assumed- excluded region) – so checking `g[mid]` there could never distinguish anything. This is moot now that the port matches the actual C algorithm (which checks `g[igneg]`, the first point genuinely outside the current gate), but is recorded here since it was the first of the two Python-side bugs found.
3. *A genuine ambiguity in SASfit's own C source.* In the "bisect" branch (triggered once a trial gate already gives a non-negative result), the original C code computes a halved `indx_max_appearent_sigma` but never resets `gate4g` between that smaller value and the previous trial boundary back to 1, nor re-points its own scanning index (`igneg`) at the new boundary – so, taken literally, the next iteration's check and solve would silently re-test the same (larger) gate configuration rather than the smaller one the halving was meant to try. A related edge case (found only by testing this Python port directly): when the two search bounds are already adjacent integers, integer division always rounds down to the lower bound, which would discard a just-proven-sufficient upper bound and collapse the search straight back to no rescaling at all. Both were fixed identically in the Python port and in `sasfit_oz_solver.c` itself (the fix applied there is included in this project's own copy of the SASfit source, with a comment marking exactly what changed and why).

With all three fixes applied, the algorithm reliably finds a small, sensible rescaling and gives genuine, verified improvement over plain MSA on a moderately-coupled test case (Figure 4: contact value improves from $-0.132$ to $-0.053$, not perfectly zero but consistent with finite grid resolution). A remaining open question, not yet resolved, is why some more strongly-coupled cases still need an unexpectedly large apparent-hard-core extension to reach a fully non-negative result – plausibly a genuine slow-convergence issue in the underlying Picard iteration at large gate

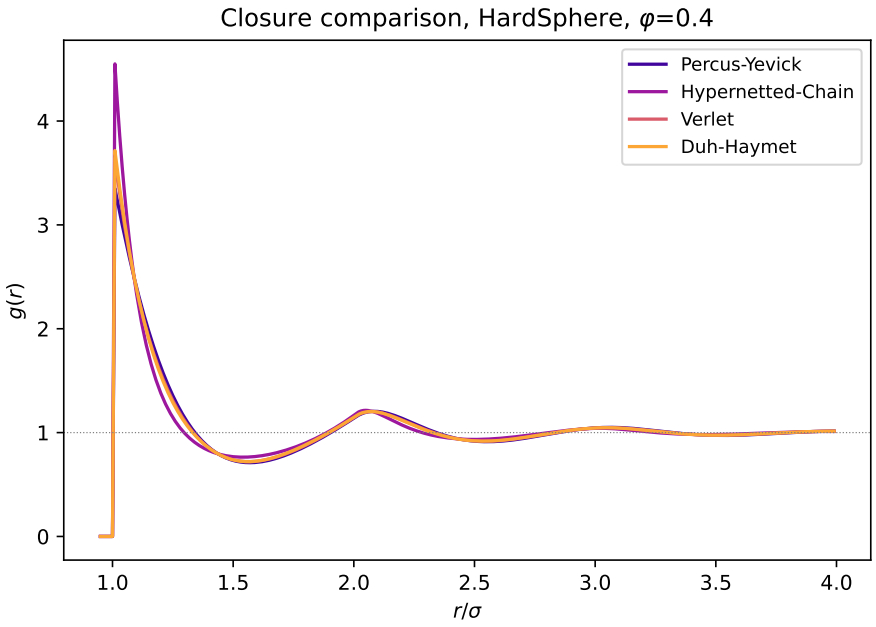


Figure 2: Closure comparison, hard spheres, $\varphi = 0.4$.

sizes (a solver-reliability question, now clearly separated from the algorithm's own correctness, which is independently validated above), though this has not been fully resolved.

### 9.4 Behaviour near difficult/critical parameter regimes

Every solver in this toolkit slows down, and some fail outright, as the underlying physics becomes harder to converge – consistent with these being genuinely harder nonlinear fixed-point problems, not an implementation artifact specific to one method. Three regimes recur throughout this project's own testing history:

- **Dense hard spheres** ($\varphi \gtrsim 0.45$, approaching random close packing): plain Picard iteration fails to converge outright; every accelerated method (Anderson, Biggs–Andrews order $\geq$1, both Newton–Krylov families) still succeeds, but needs visibly more iterations/-function evaluations than at moderate density.
- **Strongly-charged systems** (Section 9.3): plain MSA gives an unphysical negative contact value; the hand-written Anderson solver was found, in earlier testing, to converge to a *silently wrong* answer on the hardest such case tried, without raising any error – a more concerning failure mode than an honest non-convergence report, and a reminder to cross-check results from that solver specifically against an independently-validated one (e.g. `sundials4py` GMRES) for difficult states.
- **Euler–Rahman (Section 4.21)**: excluded entirely from this toolkit's offered closures, for the reasons documented there – a genuine exponential positive-feedback instability in the closure's own mathematical structure, not a solver-selection problem.

As a practical rule of thumb: if a solve fails to converge in one of these regimes, trying a different solver algorithm first (Section 6) is more likely to help than increasing `maxIterations` on the same one, since the empirical comparison of Section 6.7 shows different algorithms fail on genuinely different subsets of hard cases.

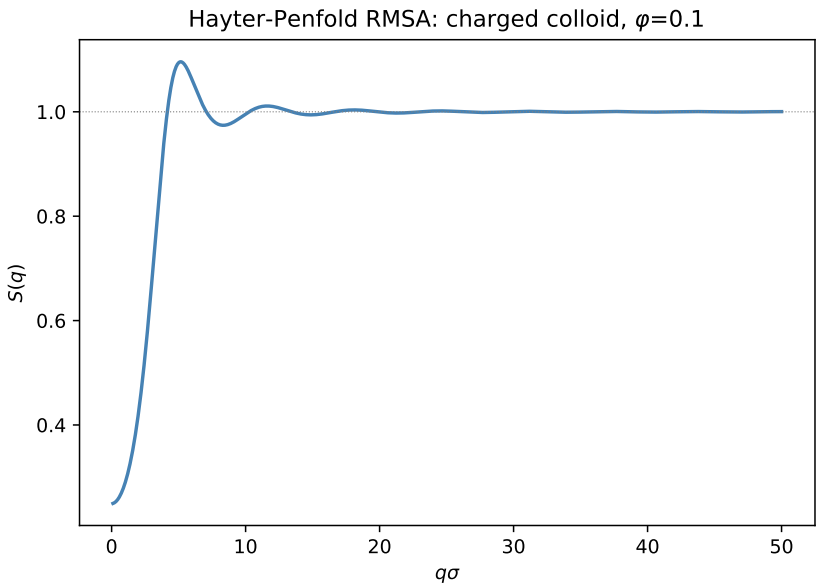


Figure 3: Hayter–Penfold RMSA structure factor for a charged colloid, $\varphi = 0.1$.

### 9.5 Speed

Figure 5 times every solver on the same moderately-hard case (hard spheres, $\varphi = 0.4$, PY closure); Section 6.7's own qualitative reliability comparison is the more important consideration in practice, but at a given level of difficulty, the SUNDIALS KIN_FP and GMRES solvers are consistently among the fastest, followed by the hand-written Anderson solver; plain Picard and Biggs–Andrews (order 0, equivalent to Picard) are consistently the slowest, as expected of unaccelerated fixed-point iteration.

Separately, Figure 6 revisits the grid-size question of Section 2: the relevant quantity for the underlying DST-based Hankel transform's own speed is $N + 1$, not $N$ itself, since the type-I DST used here is equivalent to a real FFT of size $2(N + 1)$. $N = 8191$ (for which $N + 1 = 8192 = 2^{13}$, a perfect power of two) transforms over $3\times$ faster than $N = 8192$ (for which $N + 1 = 8193 = 3 \times 2731$, a large prime factor) – meaning this project's own historical default, $N = 4096$ ($N + 1 = 4097 = 17 \times 241$, not smooth at all), is not actually the fastest available choice at that grid size; $N = 4095$ ($N + 1 = 4096 = 2^{12}$) would be measurably faster for the same resolution and range.

## Acknowledgements

Formulas and algorithms throughout this document were ported directly from SASfit's own source code (`src/sasfit_oz/`, `src/plugins/*`), with independent literature verification sought wherever practical.

## References

[1] B. C. Eu and K. Rah, *A closure for the Ornstein–Zernike relation that gives rise to the thermodynamic consistency*, J. Chem. Phys. **111**, 3327 (1999).

[2] I. Pihlajamaa and L. M. C. Janssen, *Comparison of integral equation theories of the liquid state*, Phys. Rev. E **110**, 044608 (2024); arXiv:2407.18680.

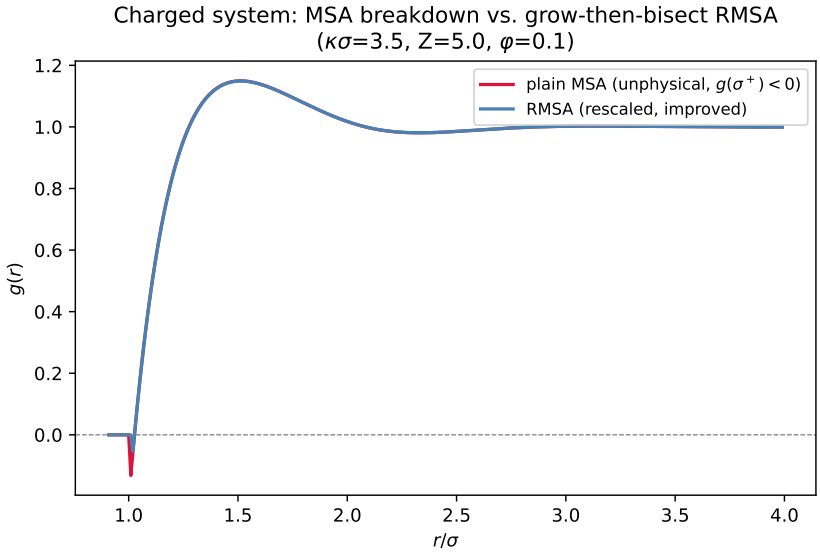


Figure 4: Plain MSA's unphysical contact value vs. this project's own grow-then-bisect RMSA rescue (post-fix), general-potential numerical solver, DLVO potential, $\kappa\sigma = 3.5$, $Z = 5$, $\varphi = 0.1$.

[3] S. K. Sharma and R. C. Sharma, *Structure factor for square-well potential in the Percus–Yevick approximation*, Physica A **89**, 213 (1977).

[4] J. B. Hayter and J. Penfold, *An analytic structure factor for macroion solutions*, Mol. Phys. **42**, 109 (1981).

[5] D. Pini, A. Parola, and L. Reatto, *A simple approximation for fluids with narrow attractive potentials*, Mol. Phys. **100**, 1507 (2002); arXiv:cond-mat/0109311.

[6] A. Santos, S. B. Yuste, and M. López de Haro, *Rational-function approximation for fluids interacting via piece-wise constant potentials*, Condens. Matter Phys. **15**, 23602 (2012).

[7] R. J. Baxter, *Percus–Yevick equation for hard spheres with surface adhesion*, J. Chem. Phys. **49**, 2770 (1968).

[8] F. J. Rogers and D. A. Young, *New, thermodynamically consistent, integral equation for simple fluids*, Phys. Rev. A **30**, 999 (1984).

[9] G. Zerah and J.-P. Hansen, *Self-consistent integral equations for fluid pair distribution functions: Another attempt*, J. Chem. Phys. **84**, 2336 (1986).

[10] L. Verlet, *Integral equations for classical fluids: I. The hard sphere case*, Mol. Phys. **41**, 183 (1980).

[11] L. Verlet, *Integral equations for classical fluids: II. Hard spheres again*, Mol. Phys. **42**, 1291 (1981).

[12] G. A. Martynov and G. N. Sarkisov, *Exact equations and the theory of liquids. V*, Mol. Phys. **49**, 1495 (1983).

[13] A. G. Vompe and G. A. Martynov, *The bridge function expansion and the self-consistency problem of the Ornstein–Zernike equation solution*, J. Chem. Phys. **100**, 5249 (1994).

[14] P. Ballone, G. Pastore, G. Galli, and D. Gazzillo, *Additive and non-additive hard sphere mixtures: Monte Carlo simulation and integral equation results*, Mol. Phys. **59**, 275 (1986).

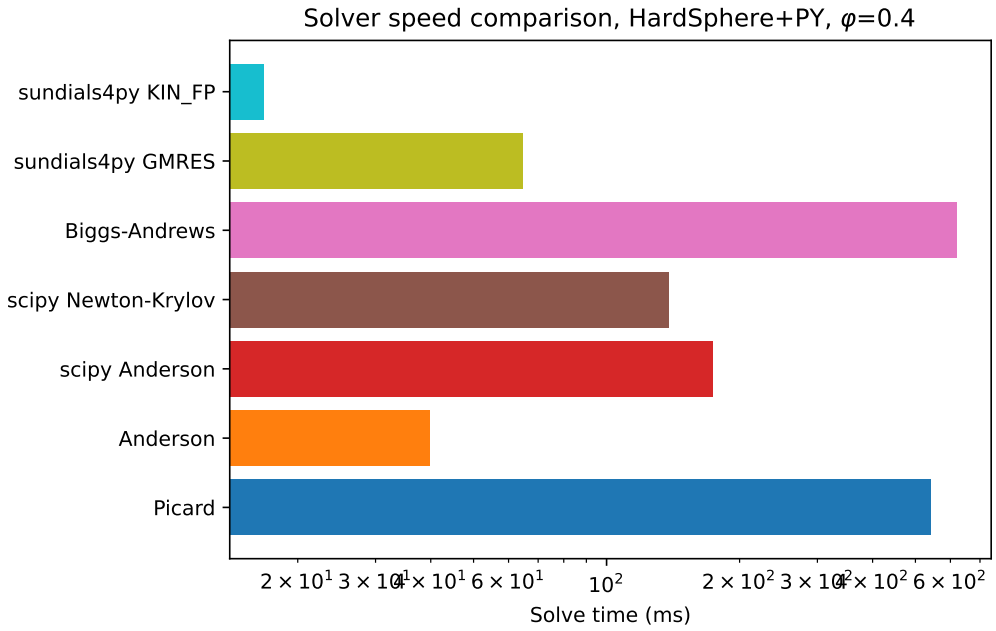


Figure 5: Solve time by algorithm, hard spheres + PY, $\varphi = 0.4$ (log scale).

[15] I. Charpentier and N. Jakse, *Exact numerical derivatives of the pair-correlation function of simple liquids using the tangent linear method*, J. Chem. Phys. **114**, 2284 (2001).

[16] J. M. Bomont and J. L. Bretonnet, *Renormalization of the indirect correlation function to extract the bridge function of simple fluids*, J. Chem. Phys. **114**, 4141 (2001).

[17] A. Kovalenko and F. Hirata, *Three-dimensional density profiles of water in contact with a solute of arbitrary shape: a RISM approach*, Chem. Phys. Lett. **290**, 237 (1998).

[18] D.-M. Duh and A. D. J. Haymet, *Integral equation theory for uncharged liquids: The Lennard-Jones fluid and the bridge function*, J. Chem. Phys. **103**, 2625 (1995).

[19] N. Choudhury and S. K. Ghosh, *Integral equation theory of Lennard-Jones fluids: A modified Verlet bridge function approach*, J. Chem. Phys. **116**, 8517 (2002).

[20] C. N. Likos and H. M. Harreis, *Star polymers: From conformations to interactions to phase diagrams*, Condens. Matter Phys. **5**, No. 1(29), 173 (2002).

[21] S. M. Oversteegen and H. N. W. Lekkerkerker, *On the accuracy of the Derjaguin approximation for depletion potentials*, Physica A **341**, 23 (2004).

[22] C. N. Likos, M. Watzlawek, and H. Löwen, *Freezing and clustering transitions for penetrable spheres*, Phys. Rev. E **58**, 3135 (1998).

[23] M. J. Fernaud, E. Lomba, and L. L. Lee, *A self-consistent integral equation study of the structure and thermodynamics of the penetrable sphere fluid*, J. Chem. Phys. **112**, 810 (2000).

[24] L. L. Lee, *An accurate integral equation theory for hard spheres: Role of the zero-separation theorems in the closure relation*, J. Chem. Phys. **103**, 9388 (1995).

[25] L. L. Lee, *Erratum: An accurate integral equation theory for hard spheres: Role of the zero-separation theorems in the closure relation*, J. Chem. Phys. **110**, 7589 (1999).

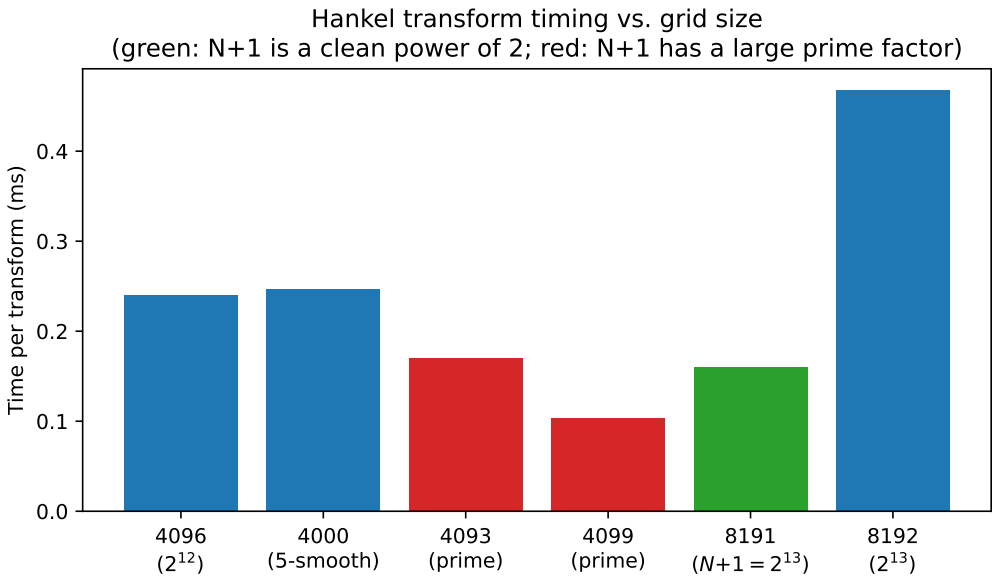


Figure 6: Hankel transform timing vs. grid size $N$: the fastest and slowest cases shown differ only in whether $N+1$ happens to be smooth, not in $N$'s own magnitude or "niceness".

[26] D. Gazzillo and A. Giacometti, *Analytic solutions for Baxter's model of sticky hard sphere fluids within closures different from the Percus–Yevick approximation*, J. Chem. Phys. **120**, 4742 (2004).

[27] Y. Rosenfeld, *Comments on the variational modified-hypernetted-chain theory for simple fluids*, J. Stat. Phys. **42**, 437 (1986).

[28] Y. Ebato and T. Miyata, *A pressure consistent bridge correction of Kovalenko–Hirata closure in Ornstein–Zernike theory for Lennard-Jones fluids by apparently adjusting sigma parameter*, AIP Advances **6**, 055111 (2016).